GoSpyder: Design and Implementation of a Network Reconnaissance Tool

Author: Aman Sharma

Year: 2024

GoSpyder is intended for authorized security testing and educational purposes only. Scanning networks or hosts without explicit permission is illegal and unethical. The author is not responsible for any misuse of this tool. Always obtain proper authorization before scanning any network or system.

ABSTRACT

GoSpyder is an advanced network scanning and service detection tool developed for network administrators, cybersecurity professionals, and researchers. Built using the Go programming language, it harnesses Go's concurrency model to perform rapid and efficient network scans across multiple hosts and ports. The key functionalities of GoSpyder include IP detection, which allows users to identify active IP addresses on a network through ping checks, port scanning to detect open ports on target hosts, service identification to determine the services running on those ports, and operating system fingerprinting using packet analysis based on Time to Live (TTL) values.

GoSpyder's architecture employs goroutines and wait groups to facilitate concurrent scanning processes, optimizing resource usage while significantly speeding up scan times. This report provides a detailed examination of GoSpyder’s design, its core components, and the algorithms powering its network scanning capabilities. By implementing efficient scanning techniques, GoSpyder meets the growing demand for enhanced cybersecurity measures in today’s complex network environments. Practical examples and case studies demonstrate the tool's utility in real-world scenarios, underscoring its importance in network analysis and security.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**IV**
	TABLE OF CONTENTS	**V**
	LIST OF FIGURES	**VIII**
	LIST OF TABLES	**X**
	LIST OF ACRONYMS	**XI**
	ORGANISATIONAL THESIS	**XII**
1	**INTRODUCTION**	**1**
	1.1 INTRODUCTION	1
	1.2 BACKGROUND	1
	1.2.1 Challenges with Existing Tools	1
	1.2.2 The GoSpyder Solution	2
	1.3 NEED FOR THE STUDY	2
	1.3.1 Efficiency	2
	1.3.2 Concurrency	3
	1.3.3 Customization	3
	1.4 OBJECTIVES	3
	1.4.1 Primary Objective	3
	1.4.2 Secondary Objective	4
	1.5 SCOPE OF WORK	5
	1.6 MOTIVATION	5
2	**LITRATURE SURVEY**	**6**
	2.1 LITERATURE SURVEY	6
3	**PROJECT DESCRIPTION**	**9**
	3.1 HARDWARE AND SOFTWARE REQUIREMENTS	9
	3.2 MODULES	9

3.2.1 Host Discovery Module 9
3.2.2 Port Scanning Module 10
3.2.3 Service Detection Module 10
3.2.4 Operating System Detection Module 10
3.2.5 Full Recon Module 11
3.2.6 User Interface Module 11
3.3 PROTOCOL USED 11

4 TOOL DEVELOPMENT – GOSPYDER 12

4.1 INTRODUCTION 12
4.2 MODULES 12
4.2.1 Host Discovery 12
4.2.2 Port Scanning 16
4.2.3 Service Detection 19
4.2.4 Operation System Detection 28
4.2.5 Full Recon 31
4.2.6 Main () Function 35

5 GENERATED OUTPUTS AND EXPLANATIONS 46

5.1 INTRODUCTION 46
5.2 HOST DISCOVERY (FUNCTIONALITY 1) 47
5.2.1 Choosing a Mode 47
5.2.2 Mode 1: Ping a Subnet 48
5.2.3 Mode 2: Ping a Single Target 49
5.3 PORT SCANNING (FUNCTIONALITY 2) 51
5.3.1 Choosing a Mode 52
5.3.2 Mode 1: Scan a Range of Ports 52
5.3.3 Mode 2: Scan Specified Ports 54
5.4 SERVICE DETECTION (FUNCTIONALITY 3) 55
5.4.1 Choosing a Mode 56
5.4.2 Mode 1: Scan a Range of Ports 57
5.4.3 Mode 2: Scan Specified Ports 59
5.5 OS DETECTION (FUNCTIONALITY 4) 61
5.6 FULL RECON (FUNCTIONALITY 5) 63

5.6.1 Choosing a Mode 64
5.6.2 Mode 1: Quick Scan 64
5.6.3 Mode 2: All Port Scan 68

6 RESULT AND CONCLUSION 73

6.1 RESULT 73
6.2 CONCLUSION 74
6.3 COMPARATIVE ANALYSIS OF GOSPYDER AND NMAP 75
6.4 FUTURE WORK 76

APPENDIX 78

REFERENCES 83

LIST OF FIGURES

FIG.NO.	FIGURE NAME	PAGE NO.
4.1	Code Snippet - pingIP() Function	13
4.2	Code Snippet - modifyIP() Function	14
4.3	Code Snippet - scanPorts() Function	16
4.4	Code Snippet - scan_Specified_Ports() Function	18
4.5	Code Snippet - detectService() Function	20
4.6	Code Snippet - readBanner() Function	21
4.7	Code Snippet - sendHTTPRequest() Function	22
4.8	Code Snippet - Multi_IP_serviceDetection() Function	24
4.9	Code Snippet - Multi_port_serviceDetection() Function	26
4.10	Code Snippet - getTTL() Function	28
4.11	Code Snippet - detectOS() Function	30
4.12	Code Snippet - Full Recon - Choosing Mode	32
4.13	Code Snippet - Full Recon - Handling Multiple Hosts	32
4.14	Code Snippet - Full Recon - Ping Check	33
4.15	Code Snippet - Full Recon - Port Scan	33
4.16	Code Snippet - Full Recon - Service Detection	33
4.17	Code Snippet - Full Recon - Operation System Detection	34
4.18	Code Snippet - Full Recon - Completion Message	34
4.19	Code Snippet - GoSpyder ASCII art	35
4.20	Code Snippet - Main() Function	36
4.21	Case 1 - Host Discovery	37
4.22	Case 2 - Port Scanning	38
4.23	Case 3 - Service Detection	39
4.24	Case 4 - OS Fingerprinting	40
4.25	Case 5 - Full Recon - Multiple Host Input	41
4.26	Case 5 - Full Recon - Ping Check	42
4.27	Case 5 - Full Recon - Port Scan	42
4.28	Case 5 - Full Recon - Service Detection	43
4.29	Case 5 - Full Recon - OS Fingerprinting	43
4.30	Goroutine Example - Port Scan	44
4.31	Error Handling Example - OS Fingerprinting	45
5.1	GoSpyder Intro Screen	46
5.2	Host Discovery Module Intro Screen	47
5.3	Choosing Mode 1 (Host Discovery)	48
5.4	Ping a Subnet mode Input (Host Discovery)	48
5.5	Ping a Subnet mode Output (Host Discovery)	49
5.6	Choosing a Mode 2 (Host Discovery)	49
5.7	Mode 2 Input (Host Discovery)	50
5.8	Mode 2 Output (Host Discovery)	50

5.9	Port Scanning Intro Screen	51
5.10	Port Scanning Functionality Input	51
5.11	Choosing Modes (Port Scanning)	52
5.12	Mode 1 Input (Port Scanning)	53
5.13	Mode 1 Output (Port Scanning)	53
5.14	Mode 2 Input (Port Scanning)	54
5.15	Mode 2 Output (Port Scanning)	55
5.16	Service Detection Intro Screen	56
5.17	Service Detection Functionality Input	56
5.18	Choosing Modes (Service Detection)	57
5.19	Mode 1 Input (Service Detection)	57
5.20	Mode 1 Output (Service Detection)	58
5.21	Mode 2 Input (Service Detection)	59
5.22	Mode 2 Output (Service Detection)	60
5.23	OS Detection Intro Screen	61
5.24	OS Detection Functionality Input	62
5.25	OS Detection Output	62
5.26	Full Recon Intro Screen	63
5.27	Choosing a Mode (Full Recon)	64
5.28	Mode 1 Input (Full Recon)	64
5.29	Mode 1 Output I (Full Recon)	65
5.30	Mode 1 Output II (Full Recon)	66
5.31	Mode 1 Output - Ping Check	66
5.32	Mode 1 Output - Port Scan	66
5.33	Mode 1 Output - Service Detection	67
5.34	Mode 1 Output - OS Detection	67
5.35	Mode 1 Output - Completion Message	68
5.36	Mode 2 Input (Full Recon)	68
5.37	Mode 2 Output I (Full Recon)	69
5.38	Mode 2 Output II (Full Recon)	70
5.39	Mode 2 Output - Ping Check	70
5.40	Mode 2 Output - Port Scan	71
5.41	Mode 2 Output - Service Detection	71
5.42	Mode 2 Output - OS Detection Scan	72
5.43	Mode 2 Output – Completion	72

LIST OF TABLES

TABLE.NO.	TABLE NAME	PAGE.NO
6.1	Comparative Analysis of GoSpyder and Nmap	82

LIST OF ACRONYMS

Go	**Golang**
OS	**Operating System**
CIDR	**Classless Inter-Domain Routing**
IP	**Internet Protocol**
TCP	**Transmission Control Protocol**
IOT	**Internet Of Things**
TTL	**Time To Live**
ICMP	**Internet Control Message Protocol**

ORGANISATIONAL THESIS

Chapter 1 Introduction

This chapter introduces the concept of network scanning and its significance in cybersecurity and network management. It discusses the challenges of traditional scanners and presents GoSpyder as a next-generation solution that leverages Go's goroutines for high-performance and concurrent network scanning, enabling faster and more efficient scans with minimal resource consumption.

Chapter 2 Literature Survey

This chapter reviews significant research and tools related to network scanning that inform the development of GoSpyder. It covers key studies on Nmap, including its principles, rules, and applications in enterprise environments, as well as innovative OS fingerprinting techniques. The chapter also discusses various modern tools like RustScan, Masscan, Hping, ZMap, Fping, and Angry IP Scanner, which offer unique capabilities for efficient port scanning, host discovery, and network analysis.

Chapter 3 Project Description

This chapter provides an overview of the hardware and software requirements essential for running GoSpyder effectively. Additionally, it includes a detailed breakdown of the tool's key modules, outlining their functionalities and contributions to the overall network scanning process.

Chapter 4 Tool Development – GoSpyder

This chapter covers the development of **GoSpyder**, a lightweight and concurrent network scanning tool. It details the key modules like host discovery, port scanning, service detection, and OS identification, built using Go's concurrency model for high performance and efficiency. Code snippets and explanations are provided to demonstrate how these functionalities were implemented.

Chapter 5 Generated Output and Explanation

This chapter reviews the outputs generated by GoSpyder during various stages of execution. Each output is analyzed and explained, showcasing GoSpyder's accuracy, detection capabilities, and performance in different network environments.

Chapter 6 Result and Conclusion

This chapter reviews the successful development of the tool, which efficiently performs host discovery, port scanning, service detection, and OS identification. The tool demonstrated accurate results, reduced scan times, and an intuitive interface, making it accessible to a wide range of users. A comparative analysis with Nmap highlights GoSpyder's ease of use and targeted functionalities, while future work suggests enhancements like advanced service detection, vulnerability assessments, stealth scanning techniques, and a potential graphical interface. Overall, GoSpyder has proven to be an effective and scalable network reconnaissance tool with room for future improvements.

Chapter 1

INTRODUCTION

1.1 Introduction

In this chapter, we introduce the concept of network scanning, emphasizing its importance in the fields of cybersecurity and network management.

1.2 Background

Network scanning is a foundational activity in the field of cybersecurity, serving as the first line of defense for administrators and security professionals. It enables these professionals to gain critical visibility into their network infrastructure, identify active devices, and locate potential security vulnerabilities that could be exploited by malicious actors. As the digital landscape evolves, the attack surface continues to expand significantly, particularly due to the proliferation of cloud environments, the Internet of Things (IoT), and remote access systems. Consequently, the importance of robust network scanning tools has never been more pronounced; they are integral to establishing proactive defense mechanisms that help organizations safeguard their data and resources.

1.2.1 Challenges with Existing Tools

Despite their importance, many existing network scanning tools face significant challenges as networks grow in size and complexity. Traditional tools may experience performance bottlenecks, struggle with scalability, or consume excessive resources, making them less ideal for environments that require frequent and comprehensive scanning. For instance, while some tools are capable of performing in-depth scans, they may do so at the cost of speed, which can delay threat detection and response times. Conversely, other tools may offer quicker scans but lack the depth necessary for thorough vulnerability assessments. This dichotomy can create operational inefficiencies, leaving networks vulnerable to attacks.

Furthermore, as organizations adopt more dynamic and distributed architectures, the limitations of traditional scanning tools become more apparent. Their inability to efficiently manage large-scale scans can lead to missed vulnerabilities, rendering them inadequate for

today's complex cybersecurity landscape. As such, there is a critical need for a solution that can address these limitations while providing comprehensive scanning capabilities.

1.2.2 The GoSpyder Solution

To overcome these challenges, GoSpyder was conceptualized as a next-generation network scanning tool that leverages the high performance and concurrency model provided by the Go programming language. One of Go's standout features is its goroutines—lightweight threads that allow for concurrent execution of functions. This capability enables GoSpyder to run multiple scanning tasks simultaneously, significantly reducing overall scan times without compromising on depth or accuracy.

By utilizing goroutines, GoSpyder can efficiently manage and execute large numbers of concurrent tasks, such as scanning multiple IP addresses and ports at the same time. This not only enhances the speed of scans but also improves the tool's ability to handle complex network environments. In addition, GoSpyder's architecture is designed to be lightweight, minimizing resource consumption while maximizing throughput, making it ideal for frequent scanning in environments that require continuous security assessments.

1.3 Need for the Study

In the realm of network security, the need for effective and efficient scanning tools has never been more pronounced. As organizations increasingly rely on digital infrastructure, the ability to identify vulnerabilities and secure network environments becomes essential. This study aims to address several critical areas where existing tools may fall short, particularly in efficiency, concurrency, and customization.

1.3.1 Efficiency

While established tools like Nmap are widely recognized for their powerful scanning capabilities, they can be hindered by slow performance, especially when scanning large networks. As network sizes continue to grow, the time required to conduct comprehensive scans can become a significant drawback. Organizations require a tool that can provide timely insights into their network security without sacrificing depth or accuracy. GoSpyder seeks to bridge this gap by optimizing scanning algorithms and leveraging Go's efficient runtime to reduce scan times, ensuring that users can quickly assess their network's security posture.

1.3.2 Concurrency

Many traditional network scanning tools struggle with scalability due to their inherently sequential processing nature. This limitation can lead to bottlenecks during scans, especially in environments with numerous hosts and ports to check. In contrast, GoSpyder harnesses the concurrency model of Go, enabling it to perform multiple operations simultaneously. This design allows for parallel processing of network requests, significantly enhancing the speed and efficiency of scans. By utilizing goroutines and channels, GoSpyder can manage large-scale scans effectively, making it suitable for diverse network architectures and environments. The concurrent nature of GoSpyder not only improves scan speed but also enhances the overall user experience by providing quicker feedback and results.

1.3.3 Customization

Different network environments have unique characteristics and requirements, necessitating a flexible approach to scanning techniques and parameters. Existing tools may offer limited customization options, restricting users to predefined settings that may not align with their specific needs. GoSpyder addresses this challenge by providing a customizable framework that allows users to tailor scanning techniques according to their preferences and network conditions. Users can adjust parameters such as port ranges, scanning methods, and timeout settings, enabling them to fine-tune the tool's behavior to achieve optimal results in various scenarios. This flexibility makes GoSpyder a versatile solution, catering to the diverse needs of network administrators and security professionals across different industries.

1.4 Objectives

1.4.1 Primary Objective

Develop a Highly Efficient Network Scanning Tool

The primary objective of GoSpyder is to create a robust and highly efficient network scanning tool capable of identifying active hosts, open ports, and services on a network. This involves leveraging modern programming techniques to ensure the tool operates swiftly and accurately. Key aspects of this objective include:

- **Active Host Discovery**: Implement algorithms that enable the tool to efficiently discover live hosts within a specified subnet or range. This involves sending ICMP echo requests (ping) and processing the responses to determine which hosts are reachable.

- **Port Scanning**: Develop a feature that scans specified ports or ranges of ports on discovered hosts. This will allow users to identify which ports are open, closed, or filtered, providing insight into potential vulnerabilities.

- **Service Detection**: Incorporate methods to detect the services running on open ports. This can include analyzing service banners or conducting version detection to provide detailed information about the software and its version running on each port.

- **Performance Optimization**: Use concurrent programming techniques, such as goroutines in Go, to enable parallel scanning of multiple hosts and ports. This ensures that the tool can handle large networks efficiently without significant delays.

- **User-Friendly Interface**: Design an intuitive command-line interface that simplifies the user experience, allowing both novice and experienced users to operate the tool effectively without needing extensive technical knowledge.

1.4.2 Secondary Objective

Provide a Framework for Expansion and Modification

The secondary objective of GoSpyder is to create a flexible framework that allows for easy expansion and modification, enabling users to add advanced scanning features as needed. This involves several key considerations:

- **Modular Design**: Structure the codebase in a modular way, where different functionalities (e.g., host discovery, port scanning, service detection) are encapsulated in separate modules. This modularity facilitates easy updates and additions to the tool's capabilities.

- **Plugin Architecture**: Consider implementing a plugin architecture that allows users to develop and integrate their own scanning features or enhancements. This can encourage community contributions and foster a collaborative environment for tool improvement.

- **Documentation and Examples**: Provide comprehensive documentation, including usage guides and examples, to help users understand how to extend the tool's

functionality. Clear instructions can reduce the barrier to entry for users who wish to customize GoSpyder for specific use cases.

- **Support for Advanced Features**: Design the framework to support the integration of advanced scanning techniques, such as OS fingerprinting and vulnerability scanning. This will ensure that the tool remains relevant and useful as network security practices evolve.

- **Community Engagement**: Encourage user feedback and community engagement to continuously improve the tool. This can include forums, issue trackers, or suggestion mechanisms that allow users to voice their needs and ideas for enhancements.

By focusing on these primary and secondary objectives, GoSpyder aims to deliver a powerful and adaptable network scanning tool that meets the needs of security professionals and network administrators alike, while also being scalable for future developments.

1.5 Scope of Work

The scope of this project encompasses the comprehensive design, development, and evaluation of a network scanning tool built using the Go programming language (Golang). This initiative is driven by the increasing demand for effective cybersecurity solutions and the need for innovative tools to enhance network security. The project aims to leverage Go's capabilities to create a lightweight, efficient, and modern alternative to existing network scanners.

1.6 Motivation

The motivation behind developing GoSpyder is to explore the potential of Go in network security applications, capitalizing on its performance, concurrency support, and simplicity. By creating a tool that is both efficient and user-friendly, this project seeks to contribute meaningfully to the field of network security, providing network administrators and security professionals with a valuable resource for safeguarding their environments.

Chapter 2

LITRATURE SURVEY

2.1 Literature Survey

1. S. Liao *et al*., **"A Comprehensive Detection Approach of Nmap: Principles, Rules and Experiments,"** *2020 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)*, Chongqing, China, 2020

   This work delves into a comprehensive approach to Nmap, outlining the principles, rules, and practical experiments behind its use. The authors focus on how Nmap functions as a powerful network scanning tool and provide insights into its application in different scenarios, making it a useful reference for understanding the technical foundation of Nmap.

2. Blake Anderson and David A. McGrew**, "OS Fingerprinting: New Techniques and a Study of Information Gain and Obfuscation,"** *Proceedings of the 19th USENIX Security Symposium*, 2010.

   This paper explores innovative techniques for operating system (OS) fingerprinting, with a focus on balancing the trade-offs between information gain and obfuscation. The authors investigate how new methods can be employed to better detect operating systems on a network, and how adversaries might attempt to obfuscate their systems to avoid detection.

3. A. Orebaugh and B. Pinkard, **"Nmap in the Enterprise: Your Guide to Network Scanning,"** *Syngress Publishing*, 2007

   This book acts as a detailed guide for enterprise users to employ Nmap for network scanning purposes. It covers various practical aspects of using Nmap in an enterprise environment, from setup to advanced scanning techniques, ensuring network security and monitoring.

4. G. F. Lyon, **"Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning,"** *Nmap Project*, 2009

   Authored by Gordon Lyon, the creator of Nmap, this official guide from the Nmap project explains network discovery and security scanning techniques. The book provides in-depth knowledge about Nmap's capabilities, making it a crucial resource for those looking to gain expert knowledge on the subject.

5. C. Zhou, **"Complete Appendix of Nmap Rules,".**

   This work presents a complete appendix of Nmap rules, available online. It compiles the technical rules that govern Nmap's operations, serving as a key reference for professionals who need to understand or modify these rules for custom scanning tasks.

6. Fyodor, **"Nmap – the Network Mapper: Github Mirror of Official SYN Repository,"** *GitHub.*

   This resource provides a Github mirror of the official SYN repository for Nmap. It highlights crucial functionalities such as port scanning, OS scanning, and service version identification, offering users a codebase that showcases Nmap's core features in action.

7. Autum (Bee-san), **"RustScan: The Modern Port Scanner,"** *GitHub.*

   RustScan introduces modern, fast scanning techniques by leveraging the power of the Rust programming language. It emphasizes speed and efficiency in port scanning, making it an ideal tool for quick and comprehensive scans in large environments.

8. Robert David Graham, **"Masscan,"** *GitHub*

   This paper discusses the workings of Masscan, a tool designed to handle large-scale network scans. Masscan is capable of processing large network blocks at high speeds, which makes it an ideal choice for massive scanning tasks, providing results in an efficient manner.

9. Salvatore Sanfilippo, **"Hping: Hping Network Tool,"** *GitHub*

   Hping is a network tool used for performing host discovery, firewall testing, and network analysis. The tool allows users to craft TCP/IP packets and analyze network responses, making it versatile for various network security and testing tasks.

10. University of Michigan, **"ZMap,"** *GitHub*

    ZMap is a tool that focuses on rapid port scanning, allowing researchers and network administrators to quickly scan the entire IPv4 address space. Its focus is on scalability and performance, providing an efficient means to conduct extensive network scans.

11. David Schweikert, **"Fping,"** *GitHub*

    Fping is a network utility designed to perform fast host discovery by sending multiple ICMP echo requests simultaneously. Its ability to handle a large number of hosts in parallel makes it a reliable tool for network administrators needing quick results.

12. Anton Keks, **"Angry IP Scanner,"** *GitHub*

    Angry IP Scanner utilizes multi-threaded processing to scan multiple ports across various IP addresses simultaneously. Its lightweight design and ease of use make it a popular choice for users seeking a fast and effective scanning tool.

Chapter 3

PROJECT DESCRIPTION

3.1 Hardware and Software Requirements

To run GoSpyder efficiently, the following specifications are recommended:

- **Hardware Requirements:**
  - **Processor:** A multi-core CPU is recommended to leverage GoSpyder's concurrency.
  - **Memory:** At least 2 GB of RAM to handle larger scans across multiple hosts.
  - **Network Interface:** The system should have access to a reliable network interface to perform accurate scans.
- **Software Requirements:**
  - **Go Programming Language:** Version 1.20 or later.
  - **Operating System:** Linux, macOS, or Windows, with network administrator privileges.

3.2 Modules

3.2.1 Host Discovery Module

This module focuses on identifying active hosts within a network. It provides two modes for host discovery:

1. Ping a Subnet: This scans a given subnet in CIDR notation and identifies all responsive hosts.
2. Ping a Single Target: This targets a single IP address to verify if the host is reachable.

- **Features**
  - Fast host detection using ICMP or TCP ping.
  - Can scan large subnets efficiently due to Go's concurrency model.

3.2.2 Port Scanning Module

The port scanning module allows users to discover open ports on target hosts. It provides two modes:

1. Scan a Range of Ports: Scans a specified range of ports (e.g., 1-1000).
2. Scan Specified Ports: Allows users to input specific ports to scan, which is useful for targeted analysis.

- **Features**
  - Performs both quick and in-depth scans.
  - Detects common services running on open ports.
  - Can handle multiple IP addresses in a single scan.

3.2.3 Service Detection Module

Once open ports are identified, this module attempts to detect services running on those ports by analyzing service banners or connection responses.

- **Features**
  - Detects services like HTTP, FTP, SSH, and more.
  - Banner grabbing to collect version details of running services.
  - Helps administrators identify outdated or vulnerable services.

3.2.4 Operating System Detection Module

This module attempts to identify the operating system of a host based on TTL (Time-to-Live) values or other network-level fingerprints.

- **Features**
  - Determines probable OS families such as Linux, Windows, or Unix-based systems.
  - Provides insights into the types of systems running in the network.

3.2.5 Full Recon Module

The Full Recon module combines all the functionalities of GoSpyder into a single comprehensive scan. It performs host discovery, port scanning, service detection, and OS fingerprinting in one go.

- **Features**
    - Provides an in-depth overview of a host's network presence.
    - Reduces the need for multiple separate scans.
    - Useful for initial reconnaissance or auditing of network segments.

3.2.6 User Interface Module

This module handles the interaction with the user. It presents an intuitive command-line interface where users can select functionalities, input data, and receive formatted outputs.

- **Features**
    - Easy-to-navigate menu system.
    - Error handling for invalid inputs.
    - Displays clear and concise scan results.

3.3 Protocol Used

GoSpyder supports multiple protocols, including:

- **ICMP (Internet Control Message Protocol):** Used for ping checks to detect if a host is alive.
- **TCP (Transmission Control Protocol):** Used for port scanning.

Chapter 4

TOOL DEVELOPMENT – GoSpyder

4.1 Introduction

This chapter provides a detailed overview of the development process behind GoSpyder, an advanced network scanning tool built using the Go programming language. The chapter outlines the core functionalities implemented within the tool, including host discovery, port scanning, service detection, and operating system identification.

GoSpyder was developed with the primary objective of creating a lightweight, efficient, and concurrent tool that addresses the limitations of traditional network scanners. By leveraging the inherent advantages of Go's concurrency model, GoSpyder is capable of performing multiple tasks simultaneously, making it highly efficient for large-scale network scanning.

4.2 Modules

GoSpyder consists of several interconnected modules, each responsible for a specific functionality. These modules work together to deliver an efficient and high-performance network scanning tool.

4.2.1 Host Discovery

In the context of network reconnaissance, host discovery is a crucial step to identify which devices are active on a network. This section discusses two key functions used in GoSpyder for host discovery: **pingIP** and **modifyIP**. These functions play an integral role in determining the responsiveness of devices and generating a list of potential IP addresses within a specified subnet.

4.2.1.1 Function: pingIP

The **pingIP()** function (figure 4.1) checks whether a specific IP address is reachable by sending a single ICMP echo request (commonly known as a "ping") and evaluating the response.

```go
func pingIP(ip string) (bool) {
	cmd := exec.Command("ping", ip, "-c", "1")
	output, err := cmd.Output()
	if err != nil {
		return false
	}
	if strings.Contains(string(output), "1 packets transmitted, 1 
received") ||
		strings.Contains(string(output), "Reply from") {
		return true
	}
	return false
}
```

Figure 4.1 : Code Snippet - pingIP() Function

Explanation

a. **Function Purpose**

- The **pingIP()** function aims to determine if a given IP address is alive and responding. It is essential for establishing connectivity before performing further scans or tests.

b. **Implementation Details**

- **Command Execution**: The function utilizes the exec.Command function from the os/exec package to execute the system's ping command. The command is structured to send one packet (-c 1).
- **Output Capture**: It captures the output of the command using cmd.Output(), which returns both the command's output and any potential errors.
- **Error Handling**: If an error occurs while executing the command (for example, if the host is unreachable), the function returns false.
- **Response Evaluation**: The output is converted to a string and checked for specific phrases indicating a successful ping:
- **"1 packets transmitted, 1 received"**: This line confirms that one packet was sent and received.
- **"Reply from"**: This indicates that a reply was received from the specified IP.
- If either condition is met, the function returns true, indicating the IP is alive. Otherwise, it returns false.

c. **Use Case**:

- The **pingIP()** function is often called in loops to verify the status of multiple IPs, which is crucial in scanning operations where determining active hosts is the first step.

4.2.1.2 Function: modifyIP

The **modifyIP()** function (figure 4.2) generates a list of valid IP addresses within a specified subnet using Classless Inter-Domain Routing (CIDR) notation.

```
func modifyIP(cidr string) ([]string, error) {
	// Parse the CIDR
	ip, ipnet, err := net.ParseCIDR(cidr)
	if err != nil {
		return nil, err
	}

	// Convert IP to a slice of bytes
	ip = ip.To4() // Ensure it's IPv4
	if ip == nil {
		return nil, fmt.Errorf("Invalid IPv4 address")
	}

	var ips []string

	// Loop to modify the last octet of the IP
	for i := 1; i <= 254; i++ { // Assuming you don't want 0 or 255
		// Make a copy of the IP slice to avoid overwriting the
original IP
		modifiedIP := make(net.IP, len(ip))
		copy(modifiedIP, ip)
		modifiedIP[3] = byte(i)

		// Ensure the modified IP is within the subnet
		if ipnet.Contains(modifiedIP) {
			ips = append(ips, modifiedIP.String())
		}
	}

	// Return all the modified IPs
	return ips, nil
}
```

Figure 4.2 : Code Snippet - modifyIP() Function

Explanation

a. **Function Purpose**:

- The **modifyIP()** function generates a list of all valid IP addresses in a specified subnet, enabling the reconnaissance tool to scan multiple devices on that network efficiently.

b. **Implementation Details**:

- **CIDR Parsing**: The function starts by parsing the provided CIDR string using net.ParseCIDR(), which returns the base IP and its network definition.
- **IPv4 Conversion**: It ensures that the IP address is in IPv4 format using ip.To4(). If not, it returns an error indicating an invalid IPv4 address.
- **IP Generation Loop**:
  - The function generates IP addresses by modifying the last octet (the fourth byte) of the base IP address, iterating from 1 to 254. This range avoids 0 (network address) and 255 (broadcast address), which are not typically assigned to hosts.
  - For each iteration, it creates a copy of the original IP and updates its last octet.
- **Subnet Validation**: The function checks whether the modified IP is contained within the specified subnet using ipnet.Contains(modifiedIP). If it is, the address is added to the list of valid IPs.
- **Return Value**: After completing the loop, the function returns the slice of valid IP addresses or an error if any occurred during execution.

c. **Use Case**:

- The **modifyIP()** function is particularly useful in reconnaissance scenarios where an entire subnet needs to be scanned. By generating a list of possible host addresses, it prepares the groundwork for subsequent network discovery operations, such as using the **pingIP()** function to check which devices are active.

By incorporating these functions into GoSpyder, network administrators and security professionals can swiftly perform reconnaissance tasks, gather essential information about network topology, and prepare for further analysis or security assessments.

4.2.2 Port Scanning

Port scanning is an essential technique in network security and administration, enabling the identification of open ports on a device. Open ports can provide valuable insights into the services running on a host and potential vulnerabilities. This section details two important functions in GoSpyder for performing port scanning:

- scanPorts()
- scan_Specified_Ports().

4.2.2.1 Function: scanPorts()

The **scanPorts()** function (figure 4.3) scans a range of ports on a specified IP address to determine which ports are open.

```
func scanPorts(ip string, Start_Range int, End_Range int) {
	fmt.Println("==================================================")
	fmt.Printf(Green+" Scanning for IP Address "+Yellow+"%s\n", ip
+Reset)
	fmt.Println("--------------------------------------------------")
	for i := Start_Range; i <= End_Range; i++ {
		address := fmt.Sprintf("%s:%d", ip, i)
		_, err := net.Dial("tcp", address)
		if err == nil {
			fmt.Printf(Green+"[+] "+Yellow+"%d"+Green+"
Port"+Yellow+" Open\n"+Reset, i)
		}
	}
	fmt.Println("==================================================\n")
}
```

Figure 4.3 : Code Snippet - scanPorts() Function

Explanation

a. **Function Purpose**:

- The **scanPorts()** function is designed to check a specified range of TCP ports on a given IP address to see which ports are open. This is critical for identifying services running on the target machine.

b. **Implementation Details**:

- **Parameters**:
  - ip: The target IP address to scan.
  - Start_Range: The starting port number for the scan.
  - End_Range: The ending port number for the scan.
- **Output Formatting**: The function begins by printing a header to indicate the start of the scan. It uses color-coded output for clarity, enhancing user experience.
- **Port Loop**: A for-loop iterates over the range of ports from Start_Range to End_Range. For each port:
  - **Address Formation**: It constructs the address string in the format ip:port.
  - **Connection Attempt**: The function attempts to establish a TCP connection to the specified address using net.Dial. If the connection is successful (i.e., err is nil), it prints a message indicating that the port is open.
  - **Conclusion**: After scanning all ports in the range, it prints a closing line to signify the end of the scan.

c. **Use Case**:

- This function is particularly useful for a broad scan when the user wants to identify all active ports within a specified range on a target device, which could be useful in security audits or network management.

4.2.2.2 Function: scan_Specified_Ports

The **scan_Specified_Ports()** function (figure 4.4) scans a specific list of ports on a given IP address to determine their status (open or closed).

```
 func scan_Specified_Ports(ip string, inputs []string) {
      fmt.Println("==================================================")
      fmt.Printf(Green+" Scanning for IP Address "+Yellow+"%s"+Reset, ip)
      fmt.Println("--------------------------------------------------")
      for _, port := range inputs  {
            address := fmt.Sprintf("%s:%s", ip, port)
            _, err := net.DialTimeout("tcp", address, 2*time.Second)
            if err == nil {
                  fmt.Printf(Green+"[+] "+Yellow+"%s"+Green+"
Port"+Yellow+" Open\n"+Reset, port)
            } else {
                  fmt. Printf(Red+"[-] "+Yellow+"%s"+Green+" Port"+Red+"
Closed\n"+Reset, port)
            }}
      fmt.Println("==================================================")}
```

Figure 4.4 : Code Snippet - scan_Specified_Ports() Function

Explanation

a. **Function Purpose**

- The **scan_Specified_Ports()** function allows users to specify a list of ports to scan on a given IP address. This is helpful for targeted scanning when the user is interested in specific services.

b. **Implementation Details**

- **Parameters**:
  i. ip: The target IP address to scan.
  ii. inputs: A slice of strings representing the ports to be scanned.
- **Output Formatting**: Similar to the scanPorts() function, it begins with a header to denote the start of the scanning process.

c. **Port Loop**: The function iterates over the inputs slice:

- **Address Formation**: Each port is formatted into the address string ip:port.
- **Connection Attempt**: It uses net.DialTimeout to attempt a TCP connection with a timeout of 2 seconds. This prevents the function from hanging if a port is unresponsive.
- **Port Status Reporting**: If the connection is successful (i.e., err is nil), it prints that the port is open. If the connection fails, it indicates that the port is closed, providing feedback to the user.

d. **Use Case**

- This function is particularly beneficial when scanning known ports of interest, such as HTTP (80), HTTPS (443), or custom application ports, allowing users to quickly determine which critical services are accessible on the target device.
- Together, they provide users with a flexible and efficient way to identify open ports and the services associated with them, enhancing the tool's utility in network security assessments and vulnerability analysis.

4.2.3 Service Detection

Service detection is a critical aspect of network security assessment. It helps identify the services running on open ports of a device, allowing for vulnerability analysis and security auditing. This section discusses several functions in GoSpyder responsible for service detection, including

- detectService()
- readBanner()
- sendHTTPRequest()
- Multi_IP_serviceDetection()
- Multi_port_serviceDetection()

4.2.3.1 Function: detectService()

The **detectService()** (figure 4.5) function determines the type of service based on the received banner from a connection.

```
func detectService(banner string) string {
	if strings.HasPrefix(banner, "220") {
		return "FTP"
	} else if strings.HasPrefix(banner, "HTTP/1.1") ||
strings.HasPrefix(banner, "HTTP/2") || strings.HasPrefix(banner,
"HTTP/1.0") {
		return "HTTP"
	} else if strings.Contains(banner, "ESMTP") ||
strings.Contains(banner, "SMTP") {
		return "SMTP"
	} else if strings.HasPrefix(banner, "5.1") ||
strings.HasPrefix(banner, "5.5") {
		return "MySQL"
	} else if strings.HasPrefix(banner, "SSH-") {
		return "SSH"
}
	return "Unknown"
}
```

Figure 4.5 : Code Snippet - detectService() Function

Explanation

a. **Function Purpose**

- The **detectService()** function analyzes a service banner to identify the protocol or service type running on a specific port. It returns a string representing the service type.

b. **Implementation Details**

- **Input**: The function accepts a banner string, which contains the response received from the service running on the port.
- **Service Identification**
- It checks the prefix of the banner string against known patterns:
  - **FTP**: Identified by the prefix "220".
  - **HTTP**: Identified by prefixes "HTTP/1.1", "HTTP/1.0", or "HTTP/2".
  - **SMTP**: Identified by the presence of "ESMTP" or "SMTP" in the banner.

- **MySQL**: Identified by prefixes "5.1" or "5.5".
- **SSH**: Identified by the prefix "SSH-".

- **Default Return**: If no known pattern matches, it returns "Unknown".

c. **Use Case**

- This function is essential for service identification in network scans, allowing security analysts to understand the types of services that are exposed on a device.

4.2.3.2 Function: readBanner

The **readBanner** function (figure 4.6) reads the banner data from a connection until it reaches the end of the header section.

```
func readBanner(reader *bufio.Reader) string {
	var banner strings.Builder
	for {
		line, err := reader.ReadString('\n')
		if err != nil {
			break
		}
		banner.WriteString(line)
		if line == "\r\n" {
			break
		}
	}
	return strings.TrimSpace(banner.String())}
```

Figure 4.6 : Code Snippet - readBanner() Function

Explanation

1. **Function Purpose**

- The **readBanner()** function reads multiple lines from a buffered reader, which typically contains the banner information sent by a service when a connection is established.

2. **Implementation Details**

- **Input**: It takes a bufio.Reader as input, which represents the network connection.

- **Looping through Lines**: The function continuously reads lines until it encounters a blank line (indicating the end of the banner):
    - It reads a line using ReadString('\n').
    - Each line is appended to a strings.Builder object for efficient string concatenation.
    - The loop breaks if a line is equal to "\r\n", indicating the end of the banner section.
- **Output**: Finally, it returns the trimmed banner string, removing any extra whitespace.

3. **Use Case**

- This function is useful for extracting the service banner from the response, which can then be analyzed to determine the service type.

4.2.3.3 Function: sendHTTPRequest

The sendHTTPRequest function sends (figure 4.7) an HTTP HEAD request to the specified IP address and retrieves the HTTP response banner.

```
func sendHTTPRequest(conn net.Conn, ip string) string {
	_, err := fmt.Fprintf(conn, "HEAD / HTTP/1.1\r\nHost: %s\r\n\r\n",
ip)
	if err != nil {
		return ""
	}
	return readBanner(bufio.NewReader(conn))}
```

Figure 4.7 : Code Snippet - sendHTTPRequest() Function

Explanation

a. **Function Purpose**

- The sendHTTPRequest() function is designed to send an HTTP HEAD request to a server to obtain the response header, which typically contains useful information about the server.

b. **Implementation Details**

- **Parameters**:
  - **conn**: The active network connection to the target server.
  - **ip**: The IP address of the target server.
- **HTTP Request**: It uses fmt.Fprintf to format and send a HEAD request to the server:
  - The request format is HEAD / HTTP/1.1, including the Host header.
- **Error Handling**: If there is an error while sending the request, it returns an empty string.
- **Response Handling**: It calls readBanner to read the response banner from the server and returns it.

c. **Use Case**

- This function is essential for retrieving the HTTP response from a web server, allowing users to analyze the server's configuration and service type.

4.2.3.4 Function: Multi_IP_serviceDetection()

The **Multi_IP_serviceDetection()** function (figure 4.8) detects services across multiple ports for a given IP address.

```
func Multi_IP_serviceDetection(ip string, startPort int, endPort int, wg
*sync.WaitGroup) {
	defer wg.Done()
	var mutex sync.Mutex
	var output strings.Builder

	// Header for each IP scan
	output.WriteString(fmt.Sprintf("=====================================
============\n"))
	output.WriteString(fmt.Sprintf(Green+" Scanning IP: "+Yellow+"%s\n"+
Reset, ip))
	output.WriteString(fmt.Sprintf("=====================================
============\n\n"))

	// Loop over the ports and perform service detection
	for port := startPort; port <= endPort; port++ {
		address := fmt.Sprintf("%s:%d", ip, port)
		conn, err := net.DialTimeout("tcp", address, 1*time.Second)
		if err != nil {
			continue // Move to the next port without exiting
		}

		// Set a read deadline and get the service banner
		conn.SetReadDeadline(time.Now().Add(2 * time.Second))
		var banner string
		if port == 80 || port == 443 || port >= 1024 {
			banner = sendHTTPRequest(conn, ip)
		} else {
			banner = readBanner(bufio.NewReader(conn))
		}

		conn.Close() // Make sure to close the connection after use

		// Process the banner
		if banner == "" {
			output.WriteString(fmt.Sprintf(Green+"[+] PORT
"+Yellow+"%d"+Green+" : Connection Successful, but "+ Red +"No
Banner"+Green+" received\n\n"+Reset, port))
		} else {
			serviceType := detectService(banner)
			lines := strings.Split(banner, "\n")
			tabbedBanner := "\t" + strings.Join(lines, "\n\t") //
Adding tabs before each line
			output.WriteString(fmt.Sprintf(Green+"[+] PORT "+ Yellow
+"%d"+Green+" : "+Yellow+"%s"+Green+" Service Detected \n    Banner: "
+Reset+ "\n%s\n ", port, serviceType, tabbedBanner))
			output.WriteString("\n")
		}
	}

	// Footer after scanning all ports for an IP
	output.WriteString("=================================================
\n")

	// Ensure printing is done sequentially to avoid interleaving
	mutex.Lock()
	fmt.Print(output.String())
	mutex.Unlock()
}
```

Figure 4.8 : Code Snippet - Multi_IP_serviceDetection() Function

Explanation

a. **Function Purpose**
   - The **Multi_IP_serviceDetection()** function facilitates the detection of services running on multiple ports for a specific IP address concurrently.

b. **Implementation Details**
   - **Parameters**
     - **ip**: The target IP address.
     - **startPort**: The starting port number for the scan.
     - **endPort**: The ending port number for the scan.
     - **wg**: A wait group to synchronize goroutines.
   - **Output Formatting**: The function initializes a strings.Builder to accumulate the output, along with printing headers for clarity.
   - **Port Loop**: The function iterates over the specified port range:
     - It attempts to establish a TCP connection using net.DialTimeout.
     - If the connection is successful, it retrieves the service banner using either an HTTP request or a banner read based on the port.
   - **Banner Processing**: After retrieving the banner, it analyzes it to determine the service type using the detectService function. The output is formatted and accumulated for sequential printing.
   - **Mutex Locking**: The function uses a mutex to ensure thread-safe printing to the console.

c. **Use Case**
   - This function is essential for scanning multiple ports on a target IP address, making it useful for comprehensive service discovery in network assessments.

4.2.3.5 Function: Multi_port_serviceDetection()

The **Multi_port_serviceDetection()** function (figure 4.9) is designed to scan a specific set of ports for a given IP address.

```
func Multi_port_serviceDetection(ip string, ports []int, wg
*sync.WaitGroup) {
	defer wg.Done()
	var mutex sync.Mutex
	var output strings.Builder

	// Header for each IP scan
	output.WriteString(fmt.Sprintf("=====================================
============\n"))
	output.WriteString(fmt.Sprintf(Green+" Scanning IP:
"+Yellow+"%s\n"+Reset, ip))
	output.WriteString(fmt.Sprintf("=====================================
============\n\n"))

	// Loop over the ports and perform service detection
	for _, port := range ports {
		address := fmt.Sprintf("%s:%d", ip, port)
		conn, err := net.DialTimeout("tcp", address, 1*time.Second)
		if err != nil {
			output.WriteString(fmt.Sprintf(Red+"[-] PORT %d is
Closed\n\n"+Reset, port))
			continue // Move to the next port without exiting
		}

		// Set a read deadline and get the service banner
		conn.SetReadDeadline(time.Now().Add(2 * time.Second))
		var banner string
		if port == 80 || port == 443 || port >= 1024 {
			banner = sendHTTPRequest(conn, ip)
		} else {
			banner = readBanner(bufio.NewReader(conn))
		}

		conn.Close() // Make sure to close the connection after use

		// Process the banner
		if banner == "" {
			output.WriteString(fmt.Sprintf(Green+"[+] PORT
"+Yellow+"%d "+Green+": Connection Successful, but "+Red+"no banner
"+Green+"received\n\n"+Reset, port))
		} else {
			serviceType := detectService(banner)
			lines := strings.Split(banner, "\n")
			tabbedBanner := "\t" + strings.Join(lines, "\n\t") //
Adding tabs before each line
			output.WriteString(fmt.Sprintf(Green+"[+] PORT
"+Yellow+"%d"+Green+" : "+Yellow+"%s"+Green+" Service Detected \n
Banner: "+Reset+"\n%s\n\n"+ Reset, port, serviceType, tabbedBanner))
		}
	}
	// Footer after scanning all ports for an IP
	output.WriteString("=================================================
\n")
	// Ensure printing is done sequentially to avoid interleaving
	mutex.Lock()
	fmt.Print(output.String())
	mutex.Unlock()
}
```

Figure 4.9 : Code Snippet - Multi_port_serviceDetection() Function

Explanation

a. **Function Purpose**

- The **Multi_port_serviceDetection()** function allows the detection of services for a specific IP address across a predefined list of ports.

b. **Implementation Details**

- **Parameters**:
  - ip: The target IP address.
  - ports: A slice of integers representing the ports to scan.
  - wg: A wait group to synchronize goroutines.
- **Output Formatting**: Similar to the previous function, it initializes a strings.Builder to format the output clearly.
- **Port Loop**: The function iterates over the provided list of ports:
  - It establishes a TCP connection and retrieves the service banner as before.
- **Banner Processing**: The function checks if a banner was received and processes it to determine the service type using detectService. The output is formatted and stored for printing.
- **Mutex Locking**: It uses a mutex to ensure that console output from concurrent scans does not interleave.

c. **Use Case**

- This function is valuable for scanning specific ports on a target IP, allowing users to focus on known service ports for more efficient network analysis.

These functions collectively contribute to GoSpyder's capability for efficient network service detection, enhancing its utility in security assessments and vulnerability scans.

4.2.4 Operation System Detection

Operating system detection is a fundamental technique in network scanning, as it allows security professionals to determine the underlying OS of a target device. GoSpyder implements OS detection using two key functions: **getTTL** and **detectOS** . These functions use the Time to Live (TTL) value obtained from a ping response to infer the target system's operating system.

4.2.4.1 Function: getTTL()

The getTTL function (figure 4.10) executes a ping command to retrieve the TTL value from a target IP address.

```
func getTTL(ip string) (int, error) {
	cmd := exec.Command("ping", ip, "-c", "1")
	output, err := cmd.Output()
	if err != nil {
		return 0, fmt.Errorf("ping failed: %w", err)
	}
	outputStr := string(output)
	re := regexp.MustCompile(ttl=(\d+))
	match := re.FindStringSubmatch(outputStr)
	if len(match) < 2 {
		return 0, fmt.Errorf("TTL value not found")
	}
	ttl, err := strconv.Atoi(match[1])
	if err != nil {
		return 0, fmt.Errorf("failed to convert TTL to integer: %w",
err)
	}
	return ttl, nil
}
```

Figure 4.10 : Code Snippet - getTTL() Function

Explanation

a. **Function Purpose**

- The getTTL function is responsible for obtaining the Time to Live (TTL) value from a device by executing a ping command to the target IP.
-

b. **Implementation Details**

- **Input**: The function takes an IP address (ip) as input.
- **Ping Command Execution**:
  - It uses exec.Command to execute the system's ping command, specifying the target IP and limiting it to one ICMP packet (-c 1).
  - The output of the ping command is captured as a byte array.
- **Output Parsing**:
  - The output is converted to a string, and a regular expression (ttl=(\d+)) is used to extract the TTL value.
  - If the regular expression doesn't find a TTL value, the function returns an error.
  - If the TTL value is found, it is converted from a string to an integer using strconv.Atoi.
- **Error Handling**:
  - The function handles various errors such as failed ping execution and failed TTL conversion.

c. **Use Case**

- This function is essential for retrieving the TTL value from a target device, which is a key indicator for determining the operating system in network scans.

4.2.4.2 Function: detectOS()

The detectOS function (figure 4.10) maps a given TTL value to a specific operating system as shown in figure 4.11.

```go
func detectOS(ttl int) string {
	switch ttl {
	case 128:
		return "Windows"
	case 64:
		return "Linux/FreeBSD/OSX/Juniper/HP-UX"
	case 255:
		return "Cisco device"
	case 254:
		return "Solaris/AIX"
	case 252:
		return "Windows Server 2003/XP"
	case 240:
		return "Novell"
	case 200:
		return "HP-UX"
	case 190, 127:
		return "MacOS"
	case 100:
		return "IBM OS/2"
	case 60:
		return "AIX"
	case 50:
		return "Windows 95/98/ME"
	case 48:
		return "BSDI"
	case 30:
		return "SunOS"
	default:
		return "Unknown OS or device"
	}
}
```

Figure 4.11 : Code Snippet - detectOS() Function

Explanation

a. **Function Purpose**

- The **detectOS()** function takes the TTL value obtained from **getTTL()** and returns a string identifying the likely operating system of the target device.

b. **Implementation Details**

- **Input**: The function takes the TTL value as an integer (ttl).
- **OS Detection Logic**:
  - The function uses a switch statement to match the TTL value against known TTL-to-OS mappings. For example:
    - TTL value 128 indicates a Windows system.

        - TTL value 64 is associated with Linux/FreeBSD/OSX/Juniper/HP-UX.
        - TTL value 255 suggests a Cisco device.
    - If no match is found, the function returns "Unknown OS or device".

c. **Use Case**

- This function is crucial for identifying the operating system based on the TTL value obtained from network probes, helping to classify the target system during network reconnaissance.

Together, these two functions form a simple but effective method for determining the operating system of a device based on network characteristics such as the TTL value, making OS detection an integral part of GoSpyder's network scanning capabilities.

4.2.5 Full Recon

The Full Recon section performs a comprehensive scan on a list of target IP addresses, including ping checks, port scanning, service detection, and operating system (OS) detection. The program allows users to choose between different scanning modes (Quick Scan or All Port Scan) and handles multiple hosts.

4.2.5.1 Choosing Scan Mode: Quick or All Port Scan

The user is first prompted to choose between scanning a range of ports from **1 to 10,000** (Quick Scan) or scanning all **65,535 ports** (All Port Scan).

The default scan mode is Quick Scan (ports 1 to 10,000). The program presents the user with options to either run a Quick Scan or switch to an All Port Scan mode. The user enters their choice, which is stored in the variable mode:
If the user chooses All Port Scan (option 2), the end_port is updated to include all ports (figure 4.12):

```
fmt.Println(Blue+" [*] Quick Scan mode scans 1 - 10000 ports range
"+Reset)
fmt.Println(Blue+" [*] All Port Scan mode scans all ports (1 - 65535)
"+Reset)
fmt.Println(Blue+"Choose a mode"+Reset)
var mode int
fmt.Print(Green+"Enter your choice: "+Reset)
fmt.Scan(&mode)
if mode == 2{
    end_port = 65535
}
```

Figure 4.12 : Code Snippet - Full Recon - Choosing Mode

4.2.5.2 Handling Multiple Host Inputs

The user is prompted to enter one or more IP addresses for scanning. The IP addresses are stored in the `MultiHosts` slice.

The user can enter multiple IP addresses until an empty input (pressing Enter without typing) stops the input process as shown in the figure 4.13:

```
var MultiHosts []string
reader := bufio.NewReader(os.Stdin)
fmt.Println(Blue+" [*] Multiple Inputs can be entered"+Reset)
fmt.Println(Blue+" [*] Press Enter on empty line to finish \n"+Reset)

for {
    fmt.Print(Green+"Input: "+Reset)
    input, _ := reader.ReadString('\n')
    input = strings.TrimSpace(input)

    if input == "" {
        break // Stop when enter is pressed with an empty input
    }MultiHosts = append(MultiHosts, input)}
```

Figure 4.13 : Code Snippet - Full Recon - Handling Multiple Hosts

4.2.5.3 Executing Reconnaissance on Each Host

For each IP address in the MultiHosts list, the recon steps are executed sequentially: Ping Check, Port Scan, Service Detection, and OS Detection.

4.2.5.4 Ping Check

The **pingIP()** function checks if the IP address is alive. If the IP responds, the scan continues; otherwise, a message is printed indicating that the device is unresponsive, and the scan exits for that host as shown in the figure 4.14.

```
alive := pingIP(ip)
if alive {
    fmt.Printf(Green+"[+] IP address "+Yellow+"%s "+Green+"is
"+Yellow+"Alive.\n"+Reset, ip)
    Alive = 1
} else {
    fmt.Printf(Red+"[-] The IP address"+Yellow+" %s "+Red+"is not
responding. The device may be turned off or blocking ping requests", ip)
    continue
}
```

Figure 4.14 : Code Snippet - Full Recon - Ping Check

If the IP is alive, the recon proceeds to the next step.

4.2.5.5 Port Scan

If the IP address is alive, the **scanPorts** function scans the IP address's open ports within the range specified by the scan mode (either Quick or All Port Scan) as shown in figure 4.15.

```
if Alive == 1 {
    fmt.Println(Magenta +" \t\tPort Scan \n"+Reset)
    scanPorts(ip, start_port, end_port)
}
```

Figure 4.15 : Code Snippet - Full Recon - Port Scan

4.2.5.6 Service Detection

Once the ports have been scanned, the **Multi_IP_serviceDetection()** function is called to detect services running on the open ports. This process is executed concurrently using Go's goroutine and sync.WaitGroup to handle multiple tasks in parallel (figure 4.16).

```
wg.Add(1)
go Multi_IP_serviceDetection(ip, start_port, end_port, &wg )
wg.Wait()
```

Figure 4.16 : Code Snippet - Full Recon - Service Detection

4.2.5.7 Operating System Detection

The final phase of the recon involves detecting the operating system (OS) running on the target IP address. The **getTTL** function retrieves the **Time To Live (TTL)** value of the target IP, which is used by the **detectOS** function to infer the OS based on known TTL values (figure 4.17).

```
ttl, err := getTTL(ip)
if err != nil {
    fmt.Printf(Red+"[-] IP Address : %s - Error getting TTL \n\n"+Reset,
ip)
}
os := detectOS(ttl)
fmt.Printf(Green+"[+] IP Address:"+Yellow+" %s "+Green+"- Detected
OS:"+Yellow+" %s "+Green+"(TTL : %d)\n"+Reset, ip, os, ttl)
```

Figure 4.17 : Code Snippet - Full Recon - Operation System Detection

This step helps in identifying the target's operating system, such as Windows, Linux, or a network device like a Cisco router.

4.2.5.8 Recon Completion

Once all the recon steps (Ping, Port Scan, Service Detection, OS Detection) are complete for an IP address, the program prints a message indicating that the Full Recon for the IP is finished (figure 4.18). If there are more IPs to scan, it proceeds to the next one.

```
if length > 0 {
    fmt.Printf(Blue+" [*] Full Recon Scan for IP Address %s is Completed
\n"+Reset, ip)
    length = length - 1 }
```

Figure 4.18 : Code Snippet - Full Recon - Completion Message

If more hosts are left in the MultiHosts list, the program continues scanning the remaining IP addresses.

The **Full Recon()** function is designed to perform a thorough scan on one or more IP addresses. The process includes:

1. **Ping Check** to verify if the host is alive.

2. **Port Scanning** to identify open ports on the host.

3. **Service Detection** to determine which services are running on the open ports.

4. **Operating System Detection** based on the TTL value.

The recon process allows users to gather detailed information about the target hosts in a network efficiently.

4.2.6 Main () Function

4.2.6.1 ASCII Art Representation

The first section of the main function includes ASCII art, which is used to enhance the visual presentation of the tool. These art sections represent the name of the tool ("GoSpyder") and various functionalities like port scanning, host discovery, service detection, OS fingerprinting, and full recon.

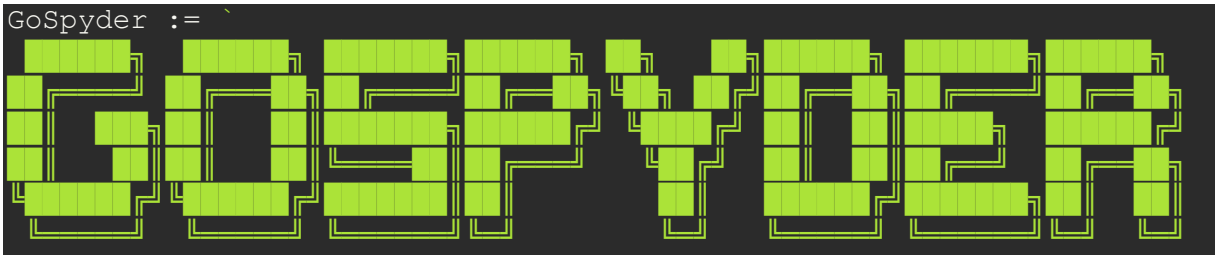

Figure 4.19 : Code Snippet - GoSpyder ASCII art

This ASCII art represents the GoSpyder banner that is printed when the program starts. It gives a sense of identity to the tool and a clear visual indication of what tool is running.
Similarly, there are ASCII arts for various operations like PortScanning, HostDiscovery, ServiceDetection, OSFingerPrinting, and FullRecon. These give users a visual cue when they are about to perform a specific operation.

4.2.6.2 Main Loop with Menu Display

The main loop in the program runs indefinitely (for {}) to repeatedly display the menu and execute the desired functionality based on user input as shown in figure 4.20.

```
for {
    showMenu()
    var choice int
    fmt.Scan(&choice)

    switch choice {
    case 1:
        // Ping Subnet or a Single Target
    case 2:
        // Port Scanning
    case 3:
        // Service Detection
    case 4:
        // OS Detection
    case 5:
        // Full Recon
    }
}
```

Figure 4.20 : Code Snippet – Main() Function

Explanation:

- showMenu() is called to display the available options in the tool.
- The fmt.Scan(&choice) reads the user's input to select an operation (like host discovery or port scanning).
- The switch statement executes the functionality based on the user's choice. Each case corresponds to a different functionality (e.g., pinging a subnet, scanning ports, detecting services, etc.).

4.2.6.3 Host Discovery (Case 1)

Host discovery is initiated when the user chooses option 1. This part of the code (figure 4.21) allows the user to either ping an entire subnet or a single target.

```
case 1:
    fmt.Println(HostDiscovery)
    fmt.Println(Blue + "Choose a Mode" + Reset)
    fmt.Println("[1] Ping a Subnet")
    fmt.Println("[2] Ping a Single Target")

    var option int
    fmt.Scan(&option)

    if option == 1 {
        fmt.Println("Enter CIDR: ")
        var cidr string
        fmt.Scan(&cidr)
        // Further logic to modify and ping all hosts in the subnet
    } else if option == 2 {
        fmt.Println("Enter IP Address: ")
        var ip string
        fmt.Scan(&ip)
        alive := pingIP(ip)
        if alive {
            fmt.Println("IP address is alive")
        } else {
            fmt.Println("IP address is not responding")
        }
    }
```

Figure 4.21 : Case 1 - Host Discovery

Explanation:

- The user is asked to choose between pinging a subnet or a single target.
- If they choose to ping a subnet, the user enters a CIDR block, and the tool pings all the IP addresses in that block.
- If they choose to ping a single target, they input an IP address, and the tool checks if that IP address is reachable using the pingIP() function.

The function pingIP() is responsible for sending a ping request and checking if the host is alive. The result is displayed accordingly.

4.2.6.4 Port Scanning (Case 2)

The second option allows the user to scan ports of a host (or multiple hosts). Users can choose between scanning a range of ports or scanning specific ports as shown in the figure 4.22.

```
case 2:
    fmt.Println(PortScanning)

    var MultiHosts []string
    reader := bufio.NewReader(os.Stdin)
    fmt.Println("Enter IP addresses of Host(s):")
    for {
        input, _ := reader.ReadString('\n')
        input = strings.TrimSpace(input)
        if input == "" {
            break
        }
        MultiHosts = append(MultiHosts, input)
    }

    fmt.Println("Choose a Mode")
    fmt.Println("[1] Scan a Range of Ports")
    fmt.Println("[2] Scan Specified Ports")

    var option int
    fmt.Scan(&option)

    if option == 1 {
        fmt.Println("Enter Starting Port:")
        var Start_Port int
        fmt.Scan(&Start_Port)
        fmt.Println("Enter Ending Port:")
        var End_Port int
        fmt.Scan(&End_Port)

        for _, ip := range MultiHosts {
            scanPorts(ip, Start_Port, End_Port)
        }
    } else if option == 2 {
        var inputs []string
        fmt.Println("Enter Ports to Scan:")
        for {
            input, _ := reader.ReadString('\n')
            input = strings.TrimSpace(input)
            if input == "" {
                break
            }
            inputs = append(inputs, input)
        }
        scan_Specified_Ports(ip, inputs)
    }
```

Figure 4.22 : Case 2 – Port Scanning

Explanation:

- The tool accepts multiple hosts for port scanning.
- Users can scan a range of ports by providing the start and end port numbers. The scanPorts() function handles this process.
- Alternatively, the user can specify individual ports, which are passed to the scan_Specified_Ports() function to check if those specific ports are open on the provided hosts.

4.2.6.5 Service Detection (Case 3)

This option scans for services running on open ports on a host or multiple hosts (figure 4.23).

```
case 3:
    fmt.Println(ServiceDectection)
    var MultiHosts []string
    reader := bufio.NewReader(os.Stdin)
    fmt.Println("Enter IP addresses of Host(s):")
    for {input, _ := reader.ReadString('\n')
        input = strings.TrimSpace(input)
        if input == "" {
            break
        }MultiHosts = append(MultiHosts, input)}
    fmt.Println("Choose a Mode")
    fmt.Println("[1] Scan a Range of Ports")
    fmt.Println("[2] Scan Specified Ports")
    var option int
    fmt.Scan(&option)
    if option == 1 {
        fmt.Println("Enter Starting Port:")
        var Start_Port int
        fmt.Scan(&Start_Port)
        fmt.Println("Enter Ending Port:")
        var End_Port int
        fmt.Scan(&End_Port)
        for _, ip := range MultiHosts {
            go Multi_IP_serviceDetection(ip, Start_Port, End_Port, &wg)}
        wg.Wait()
    } else if option == 2 {
        var Ports []int
        fmt.Println("Enter Ports to Scan:")
        for {input, _ := reader.ReadString('\n')
            input = strings.TrimSpace(input)
            if input == "" {
                break}
            port, err := strconv.Atoi(input)
            if err == nil {
                Ports = append(Ports, port)}}
        for _, ip := range MultiHosts {
            go Multi_port_serviceDetection(ip, Ports, &wg)
        }wg.Wait()}
```

Figure 4.23 : Case 3 - Service Detection

Explanation:

- Service detection scans open ports on hosts and attempts to identify the services running on those ports.
- The user can choose between scanning a range of ports or specific ports. The corresponding functions Multi_IP_serviceDetection() and Multi_port_serviceDetection() are called to detect services running on those ports.

4.2.6.6 OS Fingerprinting (Case 4)

OS fingerprinting attempts to detect the operating system of a target host based on the TTL value from ping responses as shown in the figure 4.24.

```
case 4:
    fmt.Println(OSFingerPrinting)

    var MultiHosts []string
    reader := bufio.NewReader(os.Stdin)
    fmt.Println("Enter IP addresses for OS Detection:")
    for {
        input, _ := reader.ReadString('\n')
        input = strings.TrimSpace(input)
        if input == "" {
            break
        }
        MultiHosts = append(MultiHosts, input)
    }

    for _, ip := range MultiHosts {
        ttl, err := getTTL(ip)
        if err != nil {
            fmt.Printf("Error getting TTL for IP: %s\n", ip)
            continue
        }
        os := detectOS(ttl)
        fmt.Printf("Detected OS for IP %s: %s (TTL: %d)\n", ip,
```

Figure 4.24 : Case 4 - OS Fingerprinting

Explanation

- The program prints a header for the OS fingerprinting process to the console.
- It initializes a slice to hold multiple IP addresses and creates a reader to capture user input from the console.

- The user is prompted to enter IP addresses, and the program reads the input until an empty line is entered, appending each non-empty IP to the slice.
- For each entered IP address, the program retrieves the Time To Live (TTL) value, which is useful for OS detection.
- If there's an error while retrieving the TTL, an error message is printed for that specific IP, and the program continues to the next one.
- The detected OS is determined based on the TTL value, and the program outputs the result, including the IP address, detected OS, and TTL value.

4.2.6.7 Full Recon (Case 5)

In this case, the tool performs a comprehensive reconnaissance operation that includes multiple steps: host discovery, port scanning, service detection, and OS fingerprinting. Each of these steps is crucial for gathering information about the target hosts.

1. Input Collection for Target Hosts

The user is prompted to input multiple IP addresses for the reconnaissance operation (figure 4.25).

```
var MultiHosts []string
reader := bufio.NewReader(os.Stdin)
fmt.Println("Enter IP addresses of Host(s):")
for {
    input, _ := reader.ReadString('\n')
    input = strings.TrimSpace(input)
    if input == "" {
        break
    }
    MultiHosts = append(MultiHosts, input)
}
```

Figure 4.25 : Case 5 - Full Recon - Multiple Host Input

Functionality:

- The program reads IP addresses until the user inputs a blank line, indicating they have finished providing addresses.

2. Host Discovery

The first operation in full reconnaissance is to check which of the entered hosts are alive (responding) as shown in figure 4.26.

```
fmt.Println("Performing Host Discovery...")
for _, ip := range MultiHosts {
    alive := pingIP(ip)
    if alive {
        fmt.Printf("%s is alive\n", ip)
    }
}
```

Figure 4.26 : Case 5 - Full Recon - Ping Check

Functionality:

- Each IP address in MultiHosts is passed to the pingIP() function.
- If the host is reachable (alive), the program outputs a message indicating that the host is alive.
- This step is essential because it allows the user to focus subsequent scanning efforts only on live hosts, saving time and resources.

3. Port Scanning:

After identifying which hosts are alive, the tool performs a port scan on each of these hosts.

```
fmt.Println("Performing Port Scanning...")
for _, ip := range MultiHosts {
    scanPorts(ip, 1, 65535)
}
```

Figure 4.27 : Case 5 - Full Recon - Port Scan

Functionality:

- The scanPorts() function is called for each live host, scanning all ports from 1 to 65535.
- This exhaustive scan helps identify any open ports on the host, which are potential points of access for services or vulnerabilities.

4. Service Detection:

After ports are scanned, the tool proceeds to identify the services running on those open ports (figure 4.28).

```
fmt.Println("Performing Service Detection...")
for _, ip := range MultiHosts {
    go Multi_port_serviceDetection(ip, Ports, &wg)
}
wg.Wait()
```

Figure 4.28 : Case 5 - Full Recon - Service Detection

Functionality:

- The Multi_port_serviceDetection() function is invoked for each host to determine the services associated with the open ports.
- By using goroutines (indicated by go), the tool can perform service detection concurrently across multiple hosts, making the process faster.
- wg.Wait() ensures that the program waits for all service detection goroutines to complete before moving on to the next step.

5. OS Fingerprinting:

Finally, the program attempts to determine the operating system of each host based on the TTL value returned from the ping (figure 4.29).

```
fmt.Println("Performing OS Fingerprinting...")
for _, ip := range MultiHosts {
    ttl, err := getTTL(ip)
    if err != nil {
        fmt.Printf("Error getting TTL for IP: %s\n", ip)
        continue
    }
    os := detectOS(ttl)
    fmt.Printf("Detected OS for IP %s: %s (TTL: %d)\n", ip, os, ttl)
}
```

Figure 4.29 : Case 5 - Full Recon - OS Fingerprinting

Functionality:

- The getTTL() function retrieves the Time-to-Live (TTL) value from the ping response for each IP address.

- The TTL value is then passed to the detectOS() function, which uses this value to infer the likely operating system of the target host.
- The results are printed, showing the detected operating system alongside the TTL value.

The **Full Recon** option provides users with a powerful way to gather extensive information about target hosts. By executing all four key reconnaissance steps in sequence, the tool offers insights into the following:

- **Live Hosts**: Identifies which IPs are responsive, allowing for focused analysis.
- **Open Ports**: Lists open ports that may be potential attack vectors.
- **Running Services**: Discovers services on those open ports to assess what is accessible.
- **Operating Systems**: Attempts to determine the OS to further understand the environment and potential vulnerabilities.

This comprehensive approach allows users to build a detailed profile of the target network or hosts, which is crucial for penetration testing or network analysis. The combination of these functions provides a clear picture of the network landscape, enabling users to identify possible areas of concern or opportunity for further exploration.

4.2.6.8 Concurrency with Goroutines

One of the strengths of Go is its ability to handle concurrent operations efficiently through goroutines. This is particularly useful for network operations where multiple requests may be made simultaneously. Below is how concurrency is applied in the tool, especially in the scanning and detection processes.

Example in Port Scanning

When scanning ports for multiple hosts, you use goroutines to allow parallel scanning (figure 4.30).

```
for _, ip := range MultiHosts {
    go Multi_port_serviceDetection(ip, Ports, &wg)
}
wg.Wait()
```

Figure 4.30 : Goroutine Example - Port Scan

Explanation:

- The go keyword starts a new goroutine, allowing the Multi_port_serviceDetection function to run concurrently for each IP address in MultiHosts.
- The sync.WaitGroup (wg) is used to wait for all goroutines to finish executing before the program continues. This ensures that all results are collected before moving to the next steps in the program.

4.2.6.9 Error Handling

Error handling is crucial in network applications to inform the user about any issues encountered during execution. In the code, various parts handle errors and provide feedback.

Example in OS Fingerprinting

When retrieving the TTL value, errors are managed to avoid crashing the application (Figure 4.31)

```
ttl, err := getTTL(ip)
if err != nil {
    fmt.Printf("Error getting TTL for IP: %s\n", ip)
    continue
}
```

Figure 4.31 : Error Handling Example - OS Fingerprinting

Explanation:

- The program checks if an error occurred while fetching the TTL. If an error is found, a message is printed, and the loop continues to the next IP address without interruption.
- Providing feedback helps users understand what went wrong and allows them to adjust their input or retry as needed.

Chapter 5

GENERATED OUTPUTS AND EXPLANATIONS

5.1 Introduction

This chapter provides an overview of the outputs generated by the tool **GoSpyder** during various stages of execution. Each output is analysed and explained in detail to demonstrate how the tool performs in different scenarios. The outputs offer insights into the network scanning process, detection capabilities, and overall effectiveness of the system. By reviewing these results, we can understand the accuracy, efficiency, and limitations of the tool.

The tool contains 5 modules as shown in figure 5.1:

- Host Discovery
- Port Scanning
- Service Detection
- OS Detection
- Full Recon

Figure 5.1 : GoSpyder Intro Screen

5.2 Host Discovery (Functionality 1)

The **Host Discovery** functionality allows the user to identify active hosts on the network. After selecting this functionality, the user is presented with two modes to choose from, depending on whether they want to scan an entire subnet or a specific target.

Host Discovery Functionality can be selected

5.2.1 Choosing a Mode

Once **Host Discovery** is selected, the tool prompts the user to choose between two modes (figure 5.2):

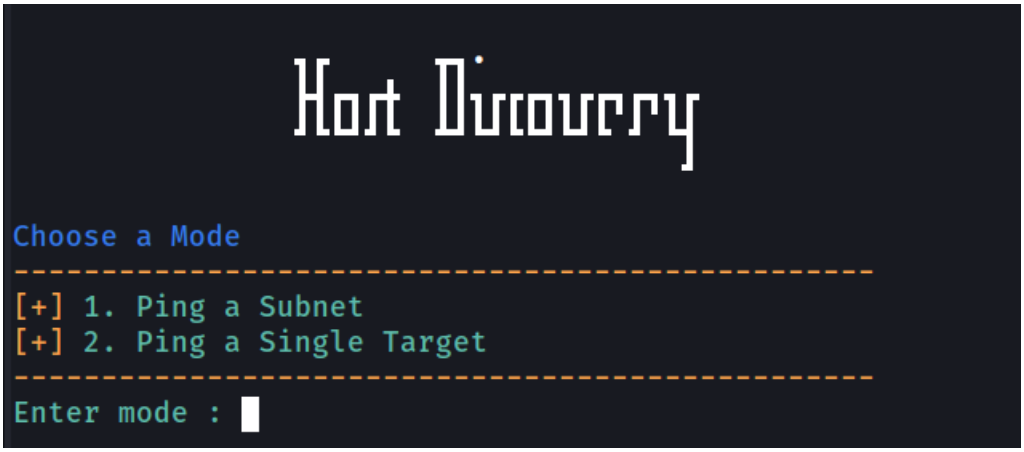

Figure 5.2 Host Discovery Module Intro Screen

- **Mode 1**: This option allows scanning an entire subnet to identify all active hosts within the given range.
- **Mode 2**: This option pings a single IP address to check if the host is active.

Once the user selects the desired mode by entering **1** or **2**, they are prompted for additional inputs based on the chosen mode.

5.2.2 Mode 1: Ping a Subnet

Description

If the user selects **Mode 1 (Ping a Subnet)** as shown in the figure 5.3, they are asked to enter the subnet in CIDR notation (e.g., 192.168.1.0/24), and the tool will scan the entire subnet to discover active hosts.

Figure 5.3 : Choosing Mode 1 (Host Discovery)

After choosing **Mode 1**, the following input is requested as shown in the figure 5.4.

Input

Figure 5.4 : Ping a Subnet mode Input (Host Discovery)

Output

```
==================================================
 List of alive hosts on the subnet 10.0.2.0/24
--------------------------------------------------
[+] IP address 10.0.2.2 is  Alive.
[+] IP address 10.0.2.3 is  Alive.
[+] IP address 10.0.2.15 is  Alive.
[+] IP address 10.0.2.4 is  Alive.
==================================================
```

Figure 5.5 : Ping a Subnet mode Output (Host Discovery)

Explanation

- The tool lists all the IP addresses that are alive within the specified subnet (**10.0.2.0/24**).
- The list shows that the hosts with the following IP addresses are active: **10.0.2.2**, **10.0.2.3**, **10.0.2.15**, and **10.0.2.4**.
- The **Alive** status confirms that the tool received responses from these IP addresses.

5.2.3 Mode 2: Ping a Single Target

Description

If the user selects **Mode 2 (Ping a Single Target)**, they are asked to provide a specific IP address to ping (figure 5.6). The tool then checks whether the target host is alive.

```
Host Discovery

Choose a Mode
--------------------------------------------------
[+] 1. Ping a Subnet
[+] 2. Ping a Single Target
--------------------------------------------------
Enter mode : 2


Enter IP address :
```

Figure 5.6 : Choosing a Mode 2 (Host Discovery)

After choosing **Mode 2**, the following input is requested as shown in the figure 5.7

Input

```
Host Discovery

Choose a Mode
-----------------------------------------------
[+] 1. Ping a Subnet
[+] 2. Ping a Single Target
-----------------------------------------------
Enter mode : 2


Enter IP address : 10.0.2.15
```

Figure 5.7 : Mode 2 Input (Host Discovery)

Output

```
===============================================
 Status of IP Adress 10.0.2.15
-----------------------------------------------
[+] IP address 10.0.2.15 is alive.
===============================================
```

Figure 5.8 : Mode 2 Output (Host Discovery)

Explanation

- The tool confirms that the specific IP address **10.0.2.15** is **alive**, meaning it successfully received a response from this host.
- This is useful for checking the status of individual devices on the network, without scanning an entire subnet.

5.3 Port Scanning (Functionality 2)

The **Port Scan** functionality allows users to scan network hosts for open or closed ports. After selecting this functionality, the user is prompted to input the IP address of the target host(s) and then choose between two scanning modes: scanning a range of ports or specifying particular ports to scan.

Upon selecting **Port Scan** by entering 2 from the main menu, the user is greeted with a prompt for IP address inputs (figure 5.9).

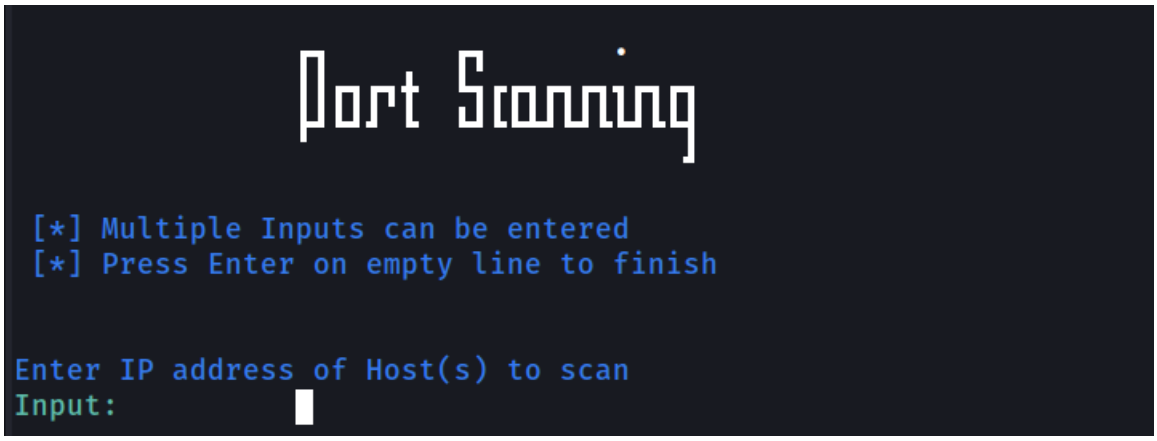

Figure 5.9 : Port Scanning Intro Screen

Input

Figure 5.10 : Port Scanning Functionality Input

After inputting the desired IP(s) as shown in the figure 5.10, the tool then asks the user to choose a port scanning mode.

5.3.1 Choosing a Mode

After entering the IP addresses, the user is prompted to choose one of the following modes for port scanning (figure 5.11):

```
Choose a Mode
-----------------------------------------------
[+] 1. Scan a Range of Ports
[+] 2. Scan Specified Ports
-----------------------------------------------
Enter your Choice :
```

Figure 5.11 : Choosing Modes (Port Scanning)

- **Mode 1**: This option allows the user to scan a continuous range of ports, defined by a starting port and an ending port.
- **Mode 2**: This option allows the user to specify particular ports to scan.

Once the user selects the desired mode, they are prompted for additional inputs based on their choice.

5.3.2 Mode 1: Scan a Range of Ports

Description

If the user selects **Mode 1 (Scan a Range of Ports)**, they are asked to provide a starting port and an ending port for the range to scan. The tool will then check all the ports within that range on the specified IP addresses to determine whether they are open.

After choosing **Mode 1**, the inputs are requested as shown in figure 5.12.

Input

```
Enter The range of ports to scan


[*] The range must be between 1 and 65535

Starting Port : 1
Ending Port : 10000
```

Figure 5.12 : Mode 1 Input (Port Scanning)

The tool will scan all ports from 1 to 10,000 on the specified host(s). Once the scan is complete, it will provide the results as shown in figure 5.13

Output

```
Enter The range of ports to scan


[*] The range must be between 1 and 65535

Starting Port : 1
Ending Port : 10000

==================================================
 Scanning for IP Address 10.0.2.15
--------------------------------------------------
[+] 21 Port Open
[+] 80 Port Open
[+] 200 Port Open
[+] 443 Port Open
==================================================
```

Figure 5.13 : Mode 1 Output (Port Scanning)

Explanation

- The scan results show that ports 21, 80, 200, and 443 on the IP address 10.0.2.15 are **open**.
- This output helps the user identify which ports are actively listening on the specified host, which can be useful for security assessments or network troubleshooting.

5.3.3 Mode 2: Scan Specified Ports

Description

If the user selects **Mode 2 (Scan Specified Ports)**, they are asked to input specific ports they wish to scan. The tool allows multiple ports to be entered, and pressing Enter on an empty line finishes the input process.

After choosing **Mode 2**, the following input is requested (figure 5.14).

Input

```
 [*] Multiple Inputs can be entered
 [*] Press Enter on empty line to finish

Enter Port(s)
Input: 80
Input: 443
Input: 900
Input: 90
Input: 200
Input:
```

Figure 5.14 : Mode 2 Input (Port Scanning)

The tool will then scan the specified ports on the entered IP address(es). Once the scan is complete, it provides the results (figure 5.15) indicating whether the specified ports are open or closed.

Output

```
==================================================
 Scanning for IP Address
--------------------------------------------------
[+] 80 Port Open
[+] 443 Port Open
[-] 900 Port Closed
[-] 90 Port Closed
[+] 200 Port Open
==================================================
```

Figure 5.15 : Mode 2 Output (Port Scanning)

Explanation

- The scan results show that ports 80, 443, and 200 are **open**, while ports 900 and 90 are **closed** on the IP address 10.0.2.15.
- This mode is useful for targeted port scans, where the user wants to check the status of specific ports rather than scanning a broad range.

5.4 Service Detection (Functionality 3)

The **Service Detection** functionality in GoSpyder allows users to detect the services running on open ports of a specified host or hosts. After providing the target IP address, the user is prompted to choose between scanning a range of ports or scanning specific ports. The tool then connects to each port and attempts to retrieve service banners or any other available information about the running service.

Upon selecting **Service Detection** by entering 3 from the main menu, the user is greeted with a prompt for entering the IP address or addresses of the hosts to scan (figure 5.16).

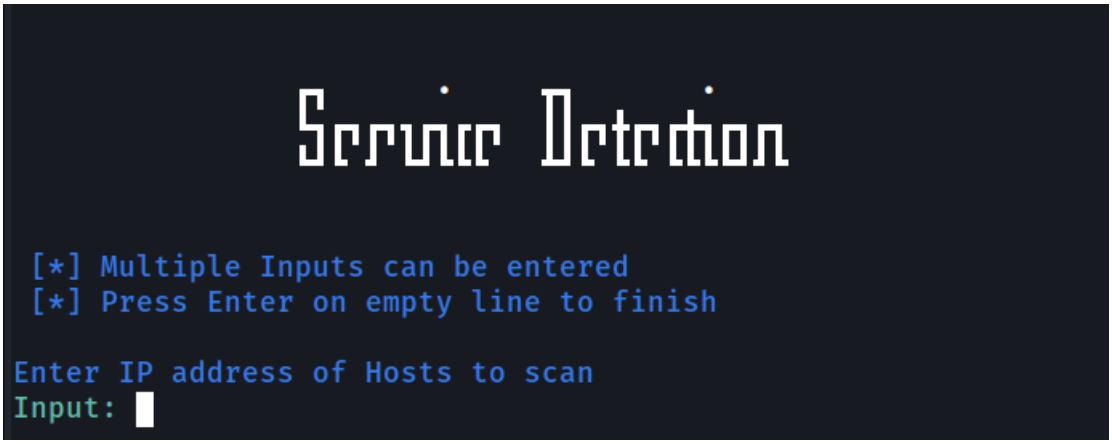

Figure 5.16 : Service Detection Intro Screen

Next, the user is prompted to input the IP address of the target host. Multiple inputs are allowed, and pressing Enter on an empty line finishes the input process (figure 5.17).

Input:

Figure 5.17 : Service Detection Functionality Input

Once the IP address has been entered, the tool prompts the user to choose a mode for port scanning.

5.4.1 Choosing a Mode

After entering the IP addresses, the user is prompted to choose one of the modes as shown in figure 5.18:

```
Choose a Mode
-------------------------------------------------
[+] 1. Scan a Range of Ports
[+] 2. Scan Specified Ports
-------------------------------------------------
Enter your Choice :
```

Figure 5.18 : Choosing Modes (Service Detection)

- **Mode 1**: Scan a range of ports by specifying a starting port and an ending port.
- **Mode 2**: Scan specific ports by manually entering them one by one.

5.4.2 Mode 1: Scan a Range of Ports

Description

If the user selects **Mode 1 (Scan a Range of Ports)**, they are prompted to input a range of ports (from the starting port to the ending port). The tool will then attempt to detect services running on all ports within the specified range.

After selecting **Mode 1**, the following inputs are required as shown in figure 5.19.

Input

```
[*] The range must be between 1 and 65535

Enter Starting Port number :1
Enter End Port number : 10000
```

Figure 5.19 : Mode 1 Input (Service Detection)

The tool scans the specified range (from port 1 to port 10,000 in this case) and attempts to detect any services running on open ports. The service banners (if available) are displayed in the output figure 5.20.

Output

```
==================================================
 Scanning IP: 10.0.2.15
==================================================

[+] PORT 21 : FTP Service Detected
    Banner:
        220 (vsFTPd 3.0.3)

[+] PORT 80 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 13:59:20 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 200 : Connection Successful, but No Banner received

[+] PORT 443 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 13:59:22 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

==================================================
```

Figure 5.20 : Mode 1 Output (Service Detection)

Explanation

- **Port 21**: The tool detected an **FTP** service running on port 21 and retrieved the banner vsFTPd 3.0.3.

- **Port 80 and 443**: The tool detected an **HTTP** service running on both ports. It retrieved the full banner, including the server software (**SimpleHTTP**), the date, content type, and content length.
- **Port 200**: Although the connection was successful, no service banner was retrieved.

This mode is useful for a broader scan where the user wants to detect services over a large range of ports, such as from port 1 to port 10,000, without needing to know the exact port numbers in advance.

5.4.3 Mode 2: Scan Specified Ports

Description

If the user selects **Mode 2 (Scan Specified Ports)**, they are prompted to manually input specific port numbers to scan. The tool will then attempt to detect services running on those particular ports.

After selecting **Mode 2**, the following inputs are required.

Input

```
 [*] Multiple Inputs can be entered
 [*] Press Enter on empty line to finish

Enter Ports to scan :
Input: 80
Input: 443
Input: 200
Input: 90
Input: 900
Input:
```

Figure 5.21 : Mode 2 Input (Service Detection)

The tool then scans only the specified ports (in this case, ports 80, 443, 200, 90, and 900) provided as shown in the figure 5.21and returns the detected services or the status of the ports as shown in figure 5.21

Output

```
==================================================
 Scanning IP: 10.0.2.15
==================================================

[+] PORT 80 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:01:33 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 443 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:01:33 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 200 : Connection Successful, but no banner received

[-] PORT 90 is Closed

[-] PORT 900 is Closed

==================================================
```

Figure 5.22 : Mode 2 Output (Service Detection)

Explanation

- **Port 80 and 443**: As in Mode 1, the tool detected an **HTTP** service on both ports, with the same banner details.
- **Port 200**: The connection was successful, but no banner was retrieved.
- **Port 90 and 900**: Both ports were found to be closed.

This mode is particularly useful when the user wants to target specific ports rather than scanning a wide range.

5.5 OS Detection (Functionality 4)

The **OS Detection** functionality in GoSpyder allows users to detect the operating system of a target host based on its network response. This detection is performed by analysing the Time to Live (TTL) value in the response packets sent from the target. Different operating systems have different TTL default values, which helps in identifying the OS.

Upon selecting **OS Detection** by entering 4 from the main menu, the user is greeted with a prompt to enter the IP address or addresses of the host(s) whose operating system is to be detected as shown in figure 5.23.

Figure 5.23 : OS Detection Intro Screen

Once the functionality is selected, the user is prompted to enter the IP address or addresses for OS detection. Multiple inputs are allowed, and the user can press Enter on an empty line to finish the input.

Input

After providing the IP address (in this case, 10.0.2.15, 0.0.0.0) as shown in figure 5.24, the tool performs an OS detection scan based on the TTL value of the response received from the target host as shown in figure 5.25

```
Enter IP Address(es)

Input: 10.0.2.15
Input: 0.0.0.0
Input:
```

Figure 5.24 : OS Detection Functionality Input

Output

```
[*] TTL means Time To Live

================================================
 OS Detection Scan
------------------------------------------------

[+] IP Address: 10.0.2.15 - Detected OS: Linux/FreeBSD/OSX/Juniper/HP-UX (TTL : 64)

[+] IP Address: 0.0.0.0 - Detected OS: Linux/FreeBSD/OSX/Juniper/HP-UX (TTL : 64)

================================================
```

Figure 5.25 : OS Detection Output

Explanation

- **IP Address**: The IP address entered by the user, in this case, 10.0.2.15 and 0.0.0.0, is shown in the output.
- **TTL (Time to Live)**: The tool uses the TTL value from the response packet to infer the operating system. Here, the TTL value is 64, which is a common default for several Unix-based systems.
- **Detected OS**: Based on the TTL value, the tool has detected that the target host is likely running one of the following operating systems:
    - **Linux**
    - **FreeBSD**
    - **OSX**
    - **Juniper/ HP-UX**

Since different systems use the same default TTL value, the detected OS is listed as one of these possibilities. In this case, the TTL value of 64 corresponds to many Unix-like operating systems, including **Linux**.

5.6 Full Recon (Functionality 5)

The **Full Recon** functionality in GoSpyder performs a comprehensive scan that includes multiple steps: a ping check, port scan, service detection, and operating system detection. This functionality offers two modes: a **Quick Scan**, which scans ports from 1 to 10000, and an **All Port Scan**, which scans all ports (1 to 65535) as shown in figure 5.26.

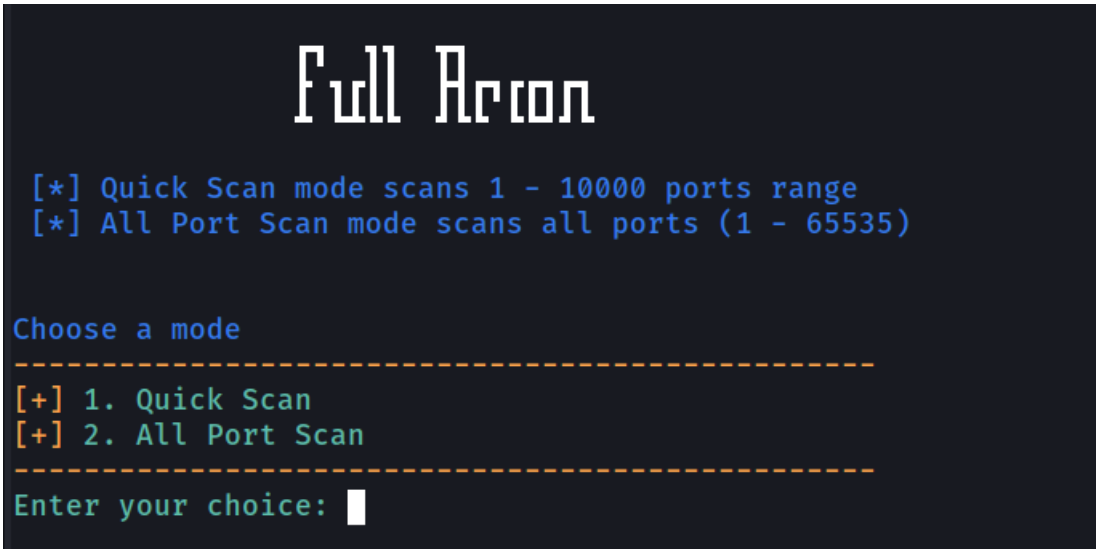

Figure 5.26 : Full Recon Intro Screen

Upon selecting the **Full Recon** functionality by entering 5 from the main menu, the user is prompted to choose between two modes.

5.6.1 Choosing a Mode

The tool then presents two scanning modes as shown in figure 5.27:

```
Choose a mode
-------------------------------------------------
[+] 1. Quick Scan
[+] 2. All Port Scan
-------------------------------------------------
Enter your choice:
```

Figure 5.27 : Choosing a Mode (Full Recon)

- **Quick Scan**, which scans ports from 1 to 10000.
- **All Port Scan**, which scans all ports (1 to 65535).

5.6.2 Mode 1: Quick Scan

The **Quick Scan** mode scans the range of ports from 1 to 10000 for each host. After selecting this mode, the user is prompted to enter the IP address or addresses to be scanned.

Input:

```
[*] Multiple Inputs can be entered
[*] Press Enter on empty line to finish

Enter IP address of Hosts to scan
Input:  10.0.2.15
Input:
```

Figure 5.28 : Mode 1 Input (Full Recon)

Once the input is provided as shown in the figure 5.28, the tool performs a full recon for the given IP address and provides the output shown in the figures 5.29 & 5.30.

Output

```
**************************************************
   Full Recon of IP Address 10.0.2.15
--------------------------------------------------


==================================================
                Ping Check
--------------------------------------------------
[+] IP address 10.0.2.15 is Alive.
==================================================

==================================================
                Port Scan
==================================================
 Scanning for IP Address 10.0.2.15
--------------------------------------------------
[+] 21 Port Open
[+] 80 Port Open
[+] 200 Port Open
[+] 443 Port Open
==================================================

==================================================
          Service Detection Scan
==================================================
 Scanning IP: 10.0.2.15
==================================================

[+] PORT 21 : FTP Service Detected
    Banner:
        220 (vsFTPd 3.0.3)

[+] PORT 80 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:30:02 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 200 : Connection Successful, but No Banner received
```

Figure 5.29 : Mode 1 Output I (Full Recon)

```
[+] PORT 443 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:30:04 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

================================================

================================================
          OS Detection Scan
------------------------------------------------
[+] IP Address: 10.0.2.15 - Detected OS: Linux/FreeBSD/OSX/Juniper/HP-UX (TTL : 64)
================================================

************************************************
 [*] Full Recon Scan for IP Address 10.0.2.15 is Completed
```

Figure 5.30 : Mode 1 Output II (Full Recon)

Explanation

- **Ping Check**: The tool performs a basic ping to check if the host is alive (figure 5.31)

```
=================================================
                Ping Check
-------------------------------------------------
[+] IP address 10.0.2.15 is Alive.
=================================================
```

Figure 5.31: Mode 1 Output - Ping Check

- **Port Scan**: The tool scans ports in the range of 1-10000 to check if they are open (figure 5.31).

```
=================================================
                Port Scan
=================================================
 Scanning for IP Address 10.0.2.15
-------------------------------------------------
[+] 21 Port Open
[+] 80 Port Open
[+] 200 Port Open
[+] 443 Port Open
=================================================
```

Figure 5.32 : Mode 1 Output - Port Scan

- **Service Detection**: The tool detects the services running on the open ports and attempts to retrieve banners where available (figure 5.33).

```
==================================================
          Service Detection Scan
==================================================
 Scanning IP: 10.0.2.15
==================================================

[+] PORT 21 : FTP Service Detected
    Banner:
        220 (vsFTPd 3.0.3)

[+] PORT 80 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:30:02 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 200 : Connection Successful, but No Banner received

[+] PORT 443 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:30:04 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

==================================================
```

Figure 5.33 : Mode 1 Output - Service Detection

- **OS Detection**: The tool performs an OS detection based on the TTL (Time to Live) value in the response packet (figure 5.34).

```
==================================================
          OS Detection Scan
--------------------------------------------------
[+] IP Address: 10.0.2.15 - Detected OS: Linux/FreeBSD/OSX/Juniper/HP-UX (TTL : 64)
==================================================
```

Figure 5.34 : Mode 1 Output - OS Detection

- Finally, the tool informs the user that the **Full Recon** scan is complete (figure 5.35).

```
**************************************************
 [*] Full Recon Scan for IP Address 10.0.2.15 is Completed
```

Figure 5.35 : Mode 1 Output - Completion Message

5.6.3 Mode 2: All Port Scan

In **All Port Scan** mode, the tool scans the entire range of ports from 1 to 65535. Similar to the **Quick Scan** mode, the user is prompted to input the IP address or addresses of the host(s) to be scanned.

```
Choose a mode
--------------------------------------------------
[+] 1. Quick Scan
[+] 2. All Port Scan
--------------------------------------------------
Enter your choice: 2


 [*] Multiple Inputs can be entered
 [*] Press Enter on empty line to finish

Enter IP address of Hosts to scan
Input:  10.0.2.15
Input:
```

Figure 5.36 : Mode 2 Input (Full Recon)

Once the input is provided, the tool performs a full recon across the entire range of ports as shown in the figure 5.36

All Port Scan Output

The output steps are similar to those of **Quick Scan**, but the **Port Scan** phase includes additional ports since all 65535 ports are scanned as show in the figures 5.37 & figure 5.38.

```
**************************************************
   Full Recon of IP Address 10.0.2.15
--------------------------------------------------


==================================================
                Ping Check
--------------------------------------------------
[+] IP address 10.0.2.15 is Alive.
==================================================

==================================================
                Port Scan
==================================================
 Scanning for IP Address 10.0.2.15
--------------------------------------------------
[+] 21 Port Open
[+] 80 Port Open
[+] 200 Port Open
[+] 443 Port Open
[+] 9000 Port Open
[+] 45000 Port Open
==================================================

==================================================
          Service Detection Scan
==================================================
 Scanning IP: 10.0.2.15
==================================================

[+] PORT 21 : FTP Service Detected
    Banner:
        220 (vsFTPd 3.0.3)

[+] PORT 80 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:34 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 200 : Connection Successful, but No Banner received
```

Figure 5.37 : Mode 2 Output I (Full Recon)

```
[+] PORT 443 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:36 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 9000 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:37 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 45000 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:37 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

==================================================

==================================================
          OS Detection Scan
--------------------------------------------------
[+] IP Address: 10.0.2.15 - Detected OS: Linux/FreeBSD/OSX/Juniper/HP-UX (TTL : 64)
==================================================

**************************************************
 [*] Full Recon Scan for IP Address 10.0.2.15 is Completed
```

Figure 5.38 : Mode 2 Output II (Full Recon)

Explanation

- **Ping Check**: The tool performs a basic ping to check if the host is alive as shown in figure 5.39.

```
==================================================
                 Ping Check
--------------------------------------------------
[+] IP address 10.0.2.15 is Alive.
==================================================
```

Figure 5.39 : Mode 2 Output - Ping Check

- **Port Scan**: The tool scans ports in the range of 1-10000 to check if they are open as shown in figure 5.40.

```
==================================================
                  Port Scan
==================================================
 Scanning for IP Address 10.0.2.15
--------------------------------------------------
[+] 21 Port Open
[+] 80 Port Open
[+] 200 Port Open
[+] 443 Port Open
[+] 9000 Port Open
[+] 45000 Port Open
==================================================
```

Figure 5.40 : Mode 2 Output - Port Scan

- **Service Detection**: The tool detects the services running on the open ports and attempts to retrieve banners where available as shown figure 5.41

```
==================================================
          Service Detection Scan
==================================================
 Scanning IP: 10.0.2.15
==================================================

[+] PORT 21 : FTP Service Detected
    Banner:
        220 (vsFTPd 3.0.3)

[+] PORT 80 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:34 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 200 : Connection Successful, but No Banner received

[+] PORT 443 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:36 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 9000 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:37 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

[+] PORT 45000 : HTTP Service Detected
    Banner:
        HTTP/1.0 200 OK
        Server: SimpleHTTP/0.6 Python/3.11.8
        Date: Fri, 27 Sep 2024 14:33:37 GMT
        Content-type: text/html; charset=utf-8
        Content-Length: 1957

==================================================
```

Figure 5.41 : Mode 2 Output - Service Detection

- **OS Detection**: The tool performs an OS detection based on the TTL (Time to Live) value in the response packet as shown in figure 5.42.

```
================================================
          OS Detection Scan
------------------------------------------------
[+] IP Address: 10.0.2.15 - Detected OS: Linux/FreeBSD/OSX/Juniper/HP-UX (TTL : 64)
================================================
```

Figure 5.43 : Mode 2 Output - OS Detection Scan

- Finally, the tool informs the user that the **Full Recon** scan is complete as shown in figure 5.43.

```
************************************************
 [*] Full Recon Scan for IP Address 10.0.2.15 is Completed
```

Figure 5.44 : Mode 2 Output – Completion

Chapter 6

RESULT AND CONCLUSION

6.1 Result

The GoSpyder tool was successfully developed to facilitate efficient and accurate network reconnaissance by integrating various essential functionalities like host discovery, port scanning, service detection, and OS identification. Each module functioned as intended, and the following results were observed during the project development phase:

- **Host Discovery**:
  - The tool was able to detect live hosts within a subnet and also provided accurate results when pinging single IP addresses.
  - Multiple test cases were run on different network configurations (e.g., /24 and /16 subnets), and the tool consistently listed all alive hosts.
- **Port Scanning**:
  - The port scanning functionality correctly identified open ports on target hosts within the specified range of common ports.
  - The speed of the scan was satisfactory with minimal network overhead.
- **Service Detection**:
  - For the identified open ports, GoSpyder was able to detect and display running services accurately.
  - This feature was validated using different network environments to ensure compatibility with a variety of network services.
- **Operating System Detection**:

- OS detection based on TCP/IP stack fingerprinting was integrated into the tool. During testing, the tool successfully identified the operating system of multiple hosts, albeit with a few limitations in cases involving certain firewall configurations.

- **Full Recon**:
  - The Full Recon mode performed a comprehensive scan, including host discovery, port scanning, service detection, and OS fingerprinting, streamlining the network reconnaissance process in a single execution.
  - The feature was tested on multiple target environments, and results confirmed the seamless integration of all the modules.
- **Performance**:
  - GoSpyder was optimized for speed and efficiency. The tool demonstrated a significant reduction in execution time compared to traditional manual methods of performing reconnaissance.

6.2 Conclusion

GoSpyder successfully achieved its primary objective of automating the network reconnaissance process in a user-friendly manner. The integration of core functionalities such as host discovery, port scanning, service detection, and OS detection into a single tool allows users to gather critical information about network infrastructure in a fast and efficient way. The modular design of GoSpyder ensures scalability and future enhancements, making it a valuable asset for network administrators and security professionals.

Key takeaways from the project include:

- **Efficiency**: The tool greatly reduces the time and effort required for network reconnaissance tasks.

- **Accuracy**: GoSpyder provided accurate and reliable results in all test cases, proving its effectiveness in real-world network environments.
- **Scalability**: The modular structure of the tool allows for easy expansion, making it adaptable to evolving network security needs.
- **Usability**: The interface and functionality of GoSpyder were designed with simplicity in mind, ensuring that users of all skill levels can utilize the tool effectively.

While GoSpyder demonstrates robust performance in typical network environments, future improvements could focus on enhancing detection in more complex scenarios (e.g., highly secured networks, systems behind advanced firewalls, etc.). Additionally, expanding the tool's capabilities to include more detailed vulnerability assessments would further increase its utility for penetration testing and security auditing.

GoSpyder is a versatile and powerful network reconnaissance tool, and its development represents a significant step toward improving the automation of network security assessments.

6.3 Comparative Analysis of GoSpyder and Nmap

Table 6.1: Comparative Analysis of GoSpyder and Nmap

Feature/Functionality	GoSpyder	Nmap
Ease of Use	User-friendly interface with clear prompts	Command-line based, may require more expertise
Multiple Input Handling	Supports multiple IP inputs in one session	Typically handles single IP or range input at a time
Focused Features	Tailored functionalities like OS detection, port scanning, and service detection are integrated	Comprehensive scanning options, but may overwhelm new users with complexity
Custom Scanning Modes	Quick Scan for efficiency; All Port Scan for thoroughness	Scanning modes are versatile but not as simplified
Simplified Output	Provides clear, structured output with explanations	Detailed but can be complex, requiring parsing

Interactive Prompts	Engaging ASCII art and prompts improve user experience	Text-based output without interactive prompts
Targeted Recon	Full recon functionality combines multiple checks in one	Requires multiple commands for full recon
Installation Size	Lightweight and minimal installation	Larger installation with additional dependencies
Documentation	Concise documentation tailored for users	Extensive documentation, but can be overwhelming
Customization	Built for specific use cases based on user feedback	Highly customizable but requires scripting knowledge

6.4 Future Work

While GoSpyder is highly functional in its current form, there are several areas for future improvement and expansion:

- **Advanced Service Detection**:
  - Currently, GoSpyder can detect services based on open ports. However, incorporating deeper service fingerprinting techniques (e.g., version detection or banner grabbing) will enhance the accuracy of service identification.
  - Expanding the database of known services and protocols could further improve detection accuracy for a wider range of network environments.
- **Vulnerability Assessment Integration**:
  - A potential future enhancement is to integrate vulnerability scanning into the tool. By cross-referencing detected services and operating systems with known vulnerabilities (e.g., CVE databases), GoSpyder could provide users with insights into potential weaknesses in their network.
  - Implementing this would turn GoSpyder from a reconnaissance tool into a more comprehensive security auditing solution.

- **Evading Network Defense Mechanisms**:
    - Future iterations could focus on bypassing or evading Intrusion Detection Systems (IDS), firewalls, and other network defense mechanisms that may hinder the accuracy of scans. Implementing stealth scanning techniques (such as packet fragmentation or slow-rate scanning) could allow GoSpyder to function more effectively in secured networks.
- **Real-Time Monitoring and Alerts**:
    - Adding real-time network monitoring functionality with active alerts could turn GoSpyder into a proactive defense tool. This would allow network administrators to receive notifications about unauthorized or suspicious activity in real time.
    - By integrating with notification platforms (such as Slack or email), GoSpyder could push critical alerts to the security team.
- **User Interface Enhancements**:
    - While GoSpyder is currently command-line based, future versions could feature a graphical user interface (GUI) to make it more accessible to users unfamiliar with command-line operations.
    - Implementing a web-based dashboard for managing and visualizing scan results in real-time could improve usability.

APPENDIX

pingIP()

```
func pingIP(ip string) (bool) {
	cmd := exec.Command("ping", ip, "-c", "1")
	output, err := cmd.Output()
	if err != nil {
		return false //, fmt.Errorf("ping failed: %w", err)
	}
	if strings.Contains(string(output), "1 packets transmitted, 1
received") ||
		strings.Contains(string(output), "Reply from") {
		return true //, nil
	}
	return false
}
```

modifyIP()

```
func modifyIP(cidr string) ([]string, error) {
	// Parse the CIDR
	ip, ipnet, err := net.ParseCIDR(cidr)
	if err != nil {
		return nil, err
	}

	// Convert IP to a slice of bytes
	ip = ip.To4() // Ensure it's IPv4
	if ip == nil {
		return nil, fmt.Errorf("Invalid IPv4 address")
	}

	var ips []string

	// Loop to modify the last octet of the IP
	for i := 1; i <= 254; i++ { // Assuming you don't want 0 or 255
		// Make a copy of the IP slice to avoid overwriting the
original IP
		modifiedIP := make(net.IP, len(ip))
		copy(modifiedIP, ip)
		modifiedIP[3] = byte(i)

		// Ensure the modified IP is within the subnet
		if ipnet.Contains(modifiedIP) {
			ips = append(ips, modifiedIP.String())
		}
	}

	// Return all the modified IPs
	return ips, nil
}
```

scanPorts(ip string, startPort, endPort int)

```
func scanPorts(ip string, Start_Range int, End_Range int) {
	fmt.Println("==================================================")
	fmt.Printf(Green+" Scanning for IP Address "+Yellow+"%s\n", ip
+Reset)
	fmt.Println("--------------------------------------------------")
	for i := Start_Range; i <= End_Range; i++ {
		address := fmt.Sprintf("%s:%d", ip, i)
		_, err := net.Dial("tcp", address)
		if err == nil {
			fmt.Printf(Green+"[+] "+Yellow+"%d"+Green+"
Port"+Yellow+" Open\n"+Reset, i)
		}
	}
	fmt.Println("==================================================\n")
}
```

scan_Specified_Ports(ip string, ports []string)

```
func scan_Specified_Ports(ip string, inputs []string) {

	fmt.Println("==================================================")
	fmt.Printf(Green+" Scanning for IP Address "+Yellow+"%s\n"+Reset, ip)
	fmt.Println("--------------------------------------------------")
	for _,port := range inputs  {
		address := fmt.Sprintf("%s:%s", ip, port)
		//fmt.Println(address)
		_, err := net.DialTimeout("tcp", address, 2*time.Second)
		if err == nil {
			fmt.Printf(Green+"[+] "+Yellow+"%s"+Green+"
Port"+Yellow+" Open\n"+Reset, port)
		} else {
			fmt.Printf(Red+"[-] "+Yellow+"%s"+Green+" Port"+Red+"
Closed\n"+Reset, port)
		}
	}
	fmt.Println("==================================================")
}
```

Multi_IP_serviceDetection()

```
func Multi_IP_serviceDetection(ip string, startPort int, endPort
int, wg *sync.WaitGroup) {
	defer wg.Done()
	var mutex sync.Mutex
	var output strings.Builder

	// Header for each IP scan
	output.WriteString(fmt.Sprintf("==============================
===================\n"))
	output.WriteString(fmt.Sprintf(Green+" Scanning IP:
"+Yellow+"%s\n"+ Reset, ip))
```

```
	output.WriteString(fmt.Sprintf("==============================
===================\n\n"))

	// Loop over the ports and perform service detection
	for port := startPort; port <= endPort; port++ {
		address := fmt.Sprintf("%s:%d", ip, port)
		conn, err := net.DialTimeout("tcp", address,
1*time.Second)
		if err != nil {
			//fmt.Print(err)
			//output.WriteString(fmt.Sprintf(Red+"[-] PORT %d :
is closed\n"+Reset, port))
			continue // Move to the next port without exiting
		}

		// Set a read deadline and get the service banner
		conn.SetReadDeadline(time.Now().Add(2 * time.Second))
		var banner string
		if port == 80 || port == 443 || port >= 1024 {
			banner = sendHTTPRequest(conn, ip)
		} else {
			banner = readBanner(bufio.NewReader(conn))
		}

		conn.Close() // Make sure to close the connection after
use

		// Process the banner
		if banner == "" {
			output.WriteString(fmt.Sprintf(Green+"[+] PORT
"+Yellow+"%d"+Green+" : Connection Successful, but "+ Red +"No
Banner"+Green+" received\n\n"+Reset, port))
		} else {

			serviceType := detectService(banner)
			lines := strings.Split(banner, "\n")
			tabbedBanner := "\t" + strings.Join(lines, "\n\t")
// Adding tabs before each line
			output.WriteString(fmt.Sprintf(Green+"[+] PORT "+
Yellow +"%d"+Green+" : "+Yellow+"%s"+Green+" Service Detected \n
Banner: " +Reset+ "\n%s\n ", port, serviceType, tabbedBanner))
			output.WriteString("\n")
		}
	}

	// Footer after scanning all ports for an IP
	//output.WriteString(fmt.Sprintf(Blue +"[+] Finished scanning:
%s\n"+Reset, ip))
	output.WriteString("==========================================
=======\n")

	// Ensure printing is done sequentially to avoid interleaving
	mutex.Lock()
	fmt.Print(output.String())
	mutex.Unlock()
}
```

Multi_port_serviceDetection ()

```
func Multi_port_serviceDetection(ip string, ports []int, wg
*sync.WaitGroup) {

	defer wg.Done()
	var mutex sync.Mutex
	var output strings.Builder

	// Header for each IP scan
	output.WriteString(fmt.Sprintf("====================================
============\n"))
	output.WriteString(fmt.Sprintf(Green+" Scanning IP:
"+Yellow+"%s\n"+Reset, ip))
	output.WriteString(fmt.Sprintf("====================================
============\n\n"))

	// Loop over the ports and perform service detection

	for _,port := range ports {
		address := fmt.Sprintf("%s:%d", ip, port)
		conn, err := net.DialTimeout("tcp", address, 1*time.Second)
		if err != nil {
			//fmt.Print(err)
			output.WriteString(fmt.Sprintf(Red+"[-] PORT %d is
Closed\n\n"+Reset, port))
			continue // Move to the next port without exiting
		}

		// Set a read deadline and get the service banner
		conn.SetReadDeadline(time.Now().Add(2 * time.Second))
		var banner string
		if port == 80 || port == 443 || port >= 1024 {
			banner = sendHTTPRequest(conn, ip)
		} else {
			banner = readBanner(bufio.NewReader(conn))
		}

		conn.Close() // Make sure to close the connection after use

		// Process the banner
		if banner == "" {
			output.WriteString(fmt.Sprintf(Green+"[+] PORT
"+Yellow+"%d "+Green+": Connection Successful, but "+Red+"no banner
"+Green+"received\n\n"+Reset, port))
		} else {
			serviceType := detectService(banner)
			lines := strings.Split(banner, "\n")
			tabbedBanner := "\t" + strings.Join(lines, "\n\t") //
Adding tabs before each line
			output.WriteString(fmt.Sprintf(Green+"[+] PORT
"+Yellow+"%d"+Green+" : "+Yellow+"%s"+Green+" Service Detected \n
Banner: "+Reset+"\n%s\n\n"+ Reset, port, serviceType, tabbedBanner))
			//output.WriteString("-----------------------------------
--------------\n")
		}
	}// Footer after scanning all ports for an IP
	//output.WriteString(fmt.Sprintf("Finished scanning: %s\n", ip))
```

```
	output.WriteString("==================================================
\n")	mutex.Lock()
	fmt.Print(output.String())
	mutex.Unlock()}
```

getTTL()

```
func getTTL(ip string) (int, error) {
	cmd := exec.Command("ping", ip, "-c", "1")
	output, err := cmd.Output()
	if err != nil {
		return 0, fmt.Errorf("ping failed: %w", err)
	}outputStr := string(output)
	re := regexp.MustCompile(`ttl=(\d+)`)
	match := re.FindStringSubmatch(outputStr)
	if len(match) < 2 {
		return 0, fmt.Errorf("TTL value not found")
	}ttl, err := strconv.Atoi(match[1])
	if err != nil {
		return 0, fmt.Errorf("failed to convert TTL to integer: %w",
err)}return ttl, nil}
```

detectOS( )

```
func detectOS(ttl int) string {
	switch ttl {
	case 128:
		return "Windows"
	case 64:
		return "Linux/FreeBSD/OSX/Juniper/HP-UX"
	case 255:
		return "Cisco device"
	case 254:
		return "Solaris/AIX"
	case 252:
		return "Windows Server 2003/XP"
	case 240:
		return "Novell"
	case 200:
		return "HP-UX"
	case 190, 127:
		return "MacOS"
	case 100:
		return "IBM OS/2"
	case 60:
		return "AIX"
	case 50:
		return "Windows 95/98/ME"
	case 48:
		return "BSDI"
	case 30:
		return "SunOS"
	default:
		return "Unknown OS or device"}}
```

REFERENCES

[1] Zhiyu Zhang and Yayu Gao, "A Comprehensive Detection Approach of Nmap: Principles, Rules and Experiments " in 2020 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC)

[2] Blake Anderson and David A. McGrew, "OS fingerprinting: New techniques and a study of information gain and obfuscation" in 2017 IEEE Conference on Communications and Network Security (CNS).

[3] A. Orebaugh and B. Pinkard, "Nmap in the enterprise: your guide to network scanning". Elsevier, 2011.

[4] G. F. Lyon, "Nmap network scanning: The official Nmap project guide to network discovery and security scanning." Insecure, 2009.

[5] C. Zhou, "Complete appendix of nmap rules,"

[6] https://nmap.org/book/man.html [Accessed on 03/06/2024]

[7] https://github.com/zmap/zmap [Accessed on 17/06/2024]

[8] https://github.com/RustScan/RustScan [Accessed on 17/06/2024]

[9] https://github.com/angryip/ipscan [Accessed on 20/06/2024]

[10] https://github.com/antirez/hping [Accessed on 29/06/2024]

[11] https://github.com/schweikert/fping [Accessed on 29/06/2024]